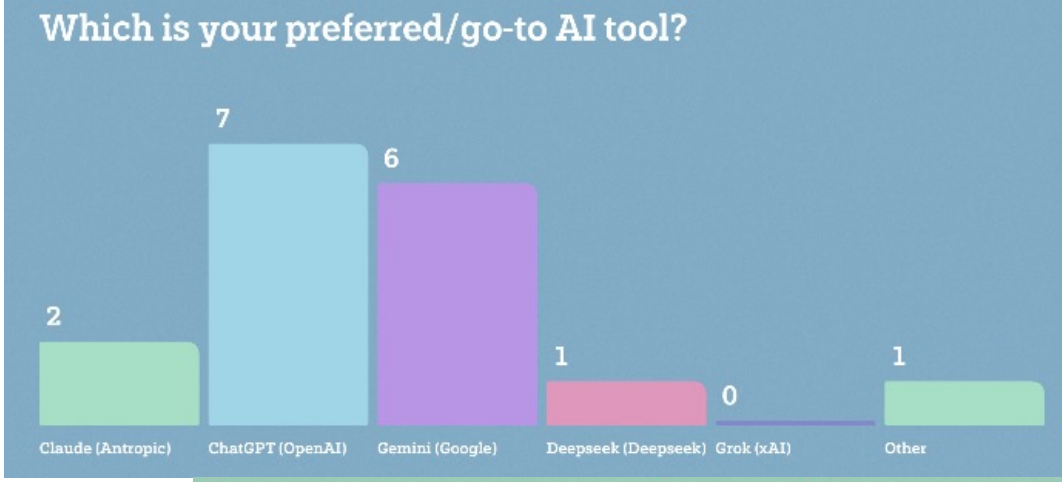
Day 2

Last Time (Tue, Jan 22nd)

- Course Overview
- ML/DL/AI Review
- Python Intro
- Airline Tweets (Lab 1)



To Do

- Discussion 1: Introductions (due today)
- Join MS Teams (code: **`w87ccaf`**)
- Lab 1 (Sat, Jan 31st)
- Kaggle 1 (Mon, Feb 2nd)
- Discussion 2 (Sat, Feb 7th)

Today

- Finish w/ Airline Tweets, and Tokenization & Vectorization
- ML Review – Models, Metrics, Neural Networks

'ELIZA,' the world's 1st chatbot, was just resurrected from 60-year-old computer code

News By Kristina Killgrove published January 17, 2025

Researchers discovered long-lost computer code and used it to resurrect the early chatbot ELIZA.

https://www.livescience.com/technology/eliza-the-worlds-1st-chatbot-was-just-resurrected-from-60-year-old-computer-code

(Recap) NLP: A language problem from scratch

Problem: You work as a data scientist for a major airline company and need to create a **sentiment analysis** model to be run on all tweets directed to your company's Twitter handle. A tweet can be considered as either a) **positive**, b) **negative**, or c) **neutral**.

(Training) Dataset: 10k Historical tweets that have been labeled by hand

	sentiment	text
0	positive	@JetBlue @JayVig I like the inflight snacks! I'm flying with you guys on 2/28! #JVMChat
1	positive	@VirginAmerica thanks guys! Sweet route over the Rockies #airplanemodewason
2	negative	@USAirways Your exchange/credit policies are worthless and shadier than the White House. Dissatisfied to the nines right now.
3	negative	@USAirways but in the meantime I'll be sleeping on a park bench on dadeland st. Thanks guys!
4	negative	@VirginAmerica hold times at call center are a bit much
5	negative	@USAirways not moving we are in the tarmac delayed for some unknown reason. I'll keep you posted
6	neutral	@JetBlue What about if I booked it through Orbitz? My email is correct, but there's a middle party.
7	negative	@united 2nd flight also delayed no pilots! But they boarded is so we can just sit here! #scheduling
8	negative	.@AmericanAir after 50 minutes on hold, and another 30 minutes on the call yes. Going to be pushing it to get to the airport on time now
9	positive	@JetBlue flight 117. proud to fly Jet Blue!

(Recap) NLP: A language problem from scratch

One Solution: Count the frequency of each word in the full dataset but separately for positive, negative, neutral tweets. We'll find a lot of stop words ☹

```
list(vocab_pos.items())[:20]
```

```
[('the', 646),
 ('to', 601),
 ('for', 460),
 ('you', 421),
 ('i', 383),
 ('@southwestair', 381),
 ('a', 350),
 ('@jetblue', 340),
 ('@united', 330),
 ('thank', 320),
 ('and', 284),
 ('thanks', 263),
 ('my', 245),
 ('@americanair', 229),
 ('in', 216),
 ('on', 215),
 ('flight', 209),
 ('of', 173),
 ('was', 160),
 ('your', 160)]
```

```
list(vocab_neg.items())[:20]
```

```
[('to', 4237),
 ('the', 2898),
 ('i', 2519),
 ('a', 2296),
 ('and', 2049),
 ('on', 1966),
 ('for', 1930),
 ('@united', 1828),
 ('my', 1699),
 ('flight', 1697),
 ('you', 1694),
 ('@usairways', 1671),
 ('is', 1483),
 ('@americanair', 1467),
 ('in', 1219),
 ('of', 1100),
 ('your', 957),
 ('not', 923),
 ('no', 896),
 ('have', 855)]
```

```
list(vocab_neu.items())[:20]
```

```
[('to', 1043),
 ('i', 723),
 ('the', 573),
 ('a', 503),
 ('on', 415),
 ('@southwestair', 414),
 ('@united', 411),
 ('you', 395),
 ('for', 380),
 ('@jetblue', 365),
 ('my', 338),
 ('flight', 337),
 ('@americanair', 293),
 ('is', 291),
 ('can', 278),
 ('in', 272),
 ('and', 254),
 ('@usairways', 233),
 ('of', 217),
 ('from', 215)]
```

(Recap) NLP: A language problem from scratch

One Solution (cont): So, let's first look at the most frequent words in the entire dataset (regardless of label value)…

```
vocab_sorted = dict(sorted(vocab.items(), key=lambda item: item[1], reverse=True))
list(vocab_sorted.items())[:25]
```

```
[('to', 5881),
 ('the', 4117),
 ('i', 3625),
 ('a', 3149),
 ('for', 2770),
 ('on', 2596),
 ('and', 2587),
 ('@united', 2569),
 ('you', 2510),
 ('my', 2282),
 ('flight', 2243),
 ('@usairways', 2061),
 ('@americanair', 1989),
 ('is', 1921),
 ('in', 1707),
 ('@southwestair', 1619),
 ('of', 1490),
 ('@jetblue', 1352),
 ('your', 1208),
 ('have', 1138),
 ('was', 1100),
 ('with', 1055),
 ('me', 1053),
 ('it', 1051),
 ('not', 1047)]
```

(Recap) NLP: A language problem from scratch

One Solution (cont): So, let's first look at the most frequent words in the entire dataset (regardless of label value)… and we removed the top k = ____ of these from the sets of positive, negative, and neutral words:

```
list(vocab_pos.items())[:25]
```

```
[('you!', 93),
 ('thanks!', 68),
 (':)', 61),
 ('best', 59),
 ('appreciate', 43),
 ('thanks.', 36),
 ('amazing', 33),
 ('awesome', 31),
 ("i'll", 28),
 ('nice', 26),
 ('keep', 25),
 ('follow', 24),
 ('well', 23),
 ('better', 23),
 ('work', 22),
 ('attendant', 22),
 ('making', 21),
 ('forward', 20),
 ('always', 20),
 ('sure', 20),
 ('sent', 20),
 ('trip', 20),
 ('care', 19),
 ('able', 19),
 ('thx', 19)]
```

```
list(vocab_neg.items())[:25]
```

```
[("won't", 107),
 ('bad', 103),
 ('days', 102),
 ('since', 101),
 ('min', 99),
 ('miss', 97),
 ('where', 95),
 ('find', 94),
 ("doesn't", 89),
 ('line', 89),
 ('give', 88),
 ('called', 86),
 ('put', 86),
 ('missed', 86),
 ('tried', 85),
 ('1', 83),
 ('customers', 83),
 ('work', 82),
 ('too', 82),
 ('5', 82),
 ('same', 82),
 ('pay', 80),
 ('into', 80),
 ('does', 79),
 ('long', 79)]
```

```
list(vocab_neu.items())[:25]
```

```
[("fleet's", 68),
 ('fleek.', 68),
 ('“@jetblue:', 52),
 ('tomorrow', 38),
 ('chance', 34),
 ('follow', 32),
 ('#destinationdragons', 31),
 ('does', 31),
 ('rt', 31),
 ('booked', 29),
 ('passengers', 29),
 ('use', 29),
 ('tickets', 29),
 ('via', 27),
 ('book', 27),
 ('sent', 27),
 ("what's", 27),
 ('think', 26),
 ('booking', 26),
 ('ceo', 25),
 ('into', 24),
 ('aa', 24),
 ('start', 22),
 ('problems', 22),
 ('send', 21)]
```

(Recap) NLP: A language problem from scratch

One Solution (cont): To classify a new tweet, let's count how many words from the positive set, negative set, and neutral set it has. The label corresponding to the set with the highest number of words, is the label we should predict.

[62]:
```
tweet2classify_i = 5555
tweet2classify = df.iloc[tweet2classify_i,:]['tokens']
df.iloc[tweet2classify_i,:]
```

[62…
```
sentiment
negative
text                                @united no- we are boarding- but why can't your agents, on the phone, taking care of 1K travell
ers, link reservations?!?!!
tokens       [@united, no-, we, are, boarding-, but, why, can't, your, agents,, on, the, phone,, taking, care, of, 1k, traveller
s,, link, reservations?!?!!]
Name: 5555, dtype: object
```

[63]:
```
pos = 0
neg = 0
neu = 0
for tok in tweet2classify:
    if tok in classifier_tokens['positive']:
        pos += 1
    elif tok in classifier_tokens['negative']:
        neg += 1
    elif tok in classifier_tokens['neutral']:
        neu += 1

print(f"pos: {pos}   neg: {neg}   neu: {neu}")
```

```
pos: 4   neg: 6   neu: 0
```

(Recap) NLP: A language problem from scratch

One Solution (cont): To see how the model worked, overall, we can classify all 10k tweets using this approach and compared the predicted labels to the actual labels. Overall accuracy: ~38% (using k =100)

```python
def predict_tweet_sentiment(tweet_tokens):
    pos = 0
    neg = 0
    neu = 0
    for tok in tweet_tokens:
        if tok in classifier_tokens['positive']:
            pos += 1
        elif tok in classifier_tokens['negative']:
            neg += 1
        elif tok in classifier_tokens['neutral']:
            neu += 1
    if pos > neg and pos > neu:
        return "positive"
    elif neu > pos and neu > neg:
        return "neutral"
    else:
        return "negative"
```

```python
df['predicted_sentiment'] = df['tokens'].apply(lambda x: predict_tweet_sentiment(x))
```

```python
df.head(10)
```

	sentiment	text	tokens	predicted_sentiment
0	positive	@JetBlue @JayVig I like the inflight snacks! I'm flying with you guys on 2/28! #JVMChat	[@jetblue, @jayvig, i, like, the, inflight, snacks!, i'm, flying, with, you, guys, on, 2/28!, #jvmchat]	positive
1	positive	@VirginAmerica thanks guys! Sweet route over the Rockies #airplanemodewason	[@virginamerica, thanks, guys!, sweet, route, over, the, rockies, #airplanemodewason]	positive
2	negative	@USAirways Your exchange/credit policies are worthless and shadier than the White House. Dissatisfied to the nines right now.	[@usairways, your, exchange/credit, policies, are, worthless, and, shadier, than, the, white, house., dissatisfied, to, the, nines, right, now.]	negative
3	negative	@USAirways but Notebook editor cells eeping on a park anks guys!	[@usairways, but, in, the, meantime, i'll, be, sleeping, on, a, park, bench, on, dadeland, st., thanks, guys!]	negative
4	negative	@VirginAmerica hold times at call center are a bit much	[@virginamerica, hold, times, at, call, center, are, a, bit, much]	positive
5	negative	@USAirways not moving we are in the tarmac delayed for some unknown reason. I'll keep you posted	[@usairways, not, moving, we, are, in, the, tarmac, delayed, for, some, unknown, reason., i'll, keep, you, posted]	positive
6	neutral	@JetBlue What about if I booked it through Orbitz? My email is correct, but there's a middle party.	[@jetblue, what, about, if, i, booked, it, through, orbitz?, my, email, is, correct,, but, there's, a, middle, party.]	negative
7	negative	@united 2nd flight also delayed no pilots! But they boarded is so we can just sit here! #scheduling	[@united, 2nd, flight, also, delayed, no, pilots!, but, they, boarded, is, so, we, can, just, sit, here!, #scheduling]	negative
8	negative	.@AmericanAir after 50 minutes on hold, and another 30 minutes on the call yes. Going to be pushing it to get to the airport on time now	[.@americanair, after, 50, minutes, on, hold,, and, another, 30, minutes, on, the, call, yes., going, to, be, pushing, it, to, get, to, the, airport, on, time, now]	negative
9	positive	@JetBlue flight 117. proud to fly Jet Blue!	[@jetblue, flight, 117., proud, to, fly, jet, blue!]	positive

(Recap) NLP: A language problem from scratch

One Solution (cont): To see how the model worked, overall, we can classify all 10k tweets using this approach and compared the predicted labels to the actual labels. Overall accuracy: ~38%

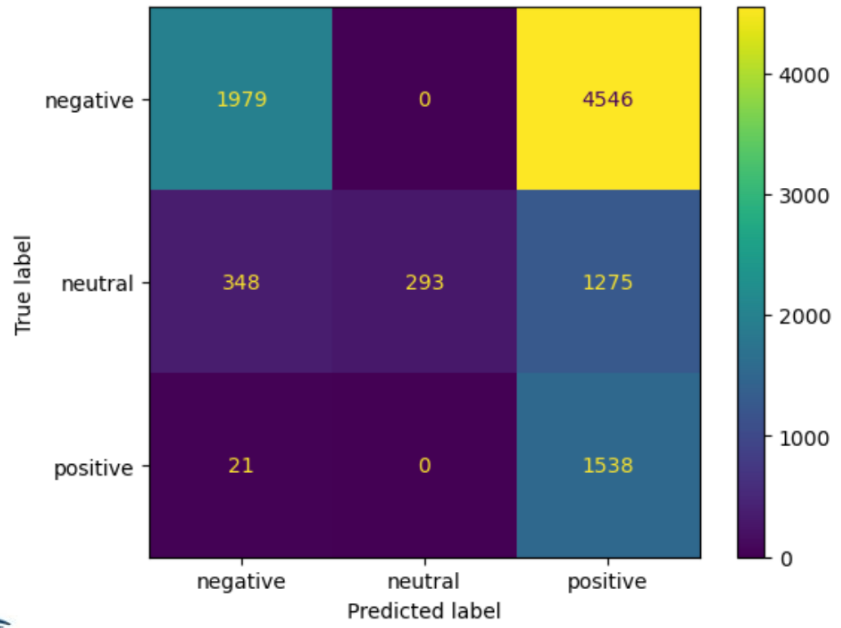
Can also use the Confusion Matrix to better understand model performance

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, accuracy
confusion_matrix(df['sentiment'], df['predicted_sentiment'])
disp = ConfusionMatrixDisplay(confusion_matrix(df['sentiment'], df['predicted_
disp.plot()

mod_accuracy = accuracy_score(df['sentiment'], df['predicted_sentiment'])
print(f"model accuracy: {mod_accuracy*100:.2f}%")
```

✓ 0.0s

```
model accuracy: 38.10%
```



Q: How do we calculate the accuracy from this?

(Recap) NLP: A language problem from scratch

Q1: Is the data good? Can the tokenization be improved?

Btw, what is the definition of **Tokenization**?

Q2: How is our "model"? Can we improve it?

Q3: Is our modeling and model evaluation process okay?
How can it be improved?

(Recap) Tokenization: Dealing with punctuation

Punctuation and special characters are easy to deal with, we just need to use

regular expressions!

Ex: *

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)    # set flag to allow verbose regexps
...     ([A-Z]\.)+        # abbreviations, e.g. U.S.A.
...   | \w+(-\w+)*        # words with optional internal hyphens
...   | \$?\d+(\.\d+)?%?  # currency and percentages, e.g. $12.40, 82%
...   | \.\.\.            # ellipsis
...   | [][.,;"'?():-_`]  # these are separate tokens; includes ], [
... '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

* https://web.stanford.edu/~jurafsky/slp3/

(Recap) Tokenization: Dealing with punctuation

Note that we'll typically rely on tools (e.g. nltk) to deal with basic tokenization issues such as this:


```
Input[1]
  from nltk.tokenize import TweetTokenizer
  tweet=u"Snow White and the Seven Degrees #MakeAMovieCold@midnight:-)"
  tokenizer = TweetTokenizer()
  print(tokenizer.tokenize(tweet.lower()))


Output[1]
  ['snow', 'white', 'and', 'the', 'seven', 'degrees',
   '#makeamoviecold', '@midnight', ':-)']
```

(Recap) Tokenization: Dealing w/ different word forms

Q: What about words that have the same meaning but are in the dataset in various inflected forms? (e.g. '*fly*' vs '*flew*' vs '*flies*', '*flown*')

(Recap) Tokenization: Dealing w/ different word forms

Q: What about words that have the same meaning but are in the dataset in various inflected forms? (e.g. '*fly*' vs '*flew*' vs '*flies*', '*flown*')

Note: Agglutinative languages combine words to create new meaning, so how should these be tokenized?

Turkish	English
kork(-mak)	(to) fear
korku	fear
korkusuz	fearless
korkusuzlaş (-mak)	(to) become fearless
korkusuzlaşmış	One who has become fearless
korkusuzlaştır(-mak)	(to) make one fearless
korkusuzlaşstırıl(-mak)	(to) be made fearless
korkusuzlaştırılmış	One who has been made fearless
korkusuzlaştırılabil(-mek)	(to) be able to be made fearless
korkusuzlaştırılabilecek	One who will be able to be made fearless
korkusuzlaştırabileceklerimiz	Ones who we can make fearless
korkusuzlaştırabileceklerimizden	From the ones who we can make fearless
korkusuzlaştırabileceklerimizdenmiş	I gather that one is one of those we can make fearless
korkusuzlaştırabileceklerimizdenmişçesine	As if that one is one of those we can make fearless
korkusuzlaştırabileceklerimizdenmişçesineyken	when it seems like that one is one of those we can make fearless

(Recap) Tokenization: Dealing w/ different word forms

Two main approaches to preprocessing text to deal with different word forms:

1. Lemmatization – attempts to identify the root form of a word and converts all inflected forms to this root form

   (advantage/strength) can deal with irregular forms/conjugations

   (disadvantage/weakness) requires use of an external database/dictionary of the language (e.g. Wordnet*)

2. Stemming – truncates words according to defined set of rules to get closer to a root form

   (advantage/strength) fast, simple, easy to understand

   (disadvantage/weakness) less robust, doesn't do a great job sometimes

* https://wordnet.princeton.edu

(Recap) Tokenization: Dealing w/ different word forms

1. Lemmatization – attempts to identify the root form of a word and converts all inflected forms to this root form, generally requires a external database (e.g. Wordnet*), and sometimes even utilizes a language model for pos tagging

2. Stemming – truncates words according to defined set of rules to get closer to a root form, can be hardcoded based on common suffixes

```
from nltk.stem import WordNetLemmatizer
# also need to run following one time on your system
# (can even be outside of this notebook)
# import nltk
# nltk.download('wordnet')
# nltk.download('omw-1.4')
lemmatizer = WordNetLemmatizer()

for w in words:
    print(w, " : ", lemmatizer.lemmatize(w, pos ="n"))
```

✓ 0.2s

```
program  :  program
programs  :  program
programming  :  programming
programmers  :  programmer
corpus  :  corpus
corpora  :  corpus
```

```
from nltk.stem import PorterStemmer
ps = PorterStemmer()

for w in words:
    print(w, " : ", ps.stem(w))
```

✓ 0.2s

```
program  :  program
programs  :  program
programming  :  program
programmers  :  programm
corpus  :  corpu
corpora  :  corpora
```

* https://wordnet.princeton.edu

Tokenization: Summary

The purpose of **tokenizing** the data is to allow us to produce a clean set of words that we can use to then create a **vocabulary/dictionary**

	sentiment	text	tokens_raw	tokens
5464	positive	@VirginAmerica first time flying Virgin, went to #SanFrancisco .Thanks for the smooth ride. Easily my new fav airline!	[@virginamerica, first, time, flying, virgin, ,, went, to, #sanfrancisco, ., thanks, for, the, smooth, ride, ., easily, my, new, fav, airline, !]	[@virginamerica, first, time, fly, virgin, ,, go, to, #sanfrancisco, ., thank, for, the, smooth, ride, ., easily, my, new, fav, airline, !]

Tokenization: Summary

The purpose of **tokenizing** the data is to allow us to produce a clean set of words that we can use to then create a **vocabulary/dictionary**

	sentiment	text	tokens_raw	tokens
5464	positive	@VirginAmerica first time flying Virgin, went to #SanFrancisco .Thanks for the smooth ride. Easily my new fav airline!	[@virginamerica, first, time, flying, virgin, ,, went, to, #sanfrancisco, ., thanks, for, the, smooth, ride, ., easily, my, new, fav, airline, !]	[@virginamerica, first, time, fly, virgin, ,, go, to, #sanfrancisco, ., thank, for, the, smooth, ride, ., easily, my, new, fav, airline, !]

Terms/definitions sometimes used:

Vocabulary – the unique set of tokens (typically words) after dealing with punctuation, removing stop words, and lemmatizing/stemming content words

Stop words – articles and prepositions (e.g. 'the', 'by', etc.) that serve mostly a grammatical purpose, and which act like fillers to hold the content words

Content words – words that carry the meaning of a piece of text that we will keep to create the vocabulary

Types – unique tokens we keep to create the vocabulary

Tokenization: Summary

The purpose of **tokenizing** the data is to allow us to produce a clean set of words that we can use to then create a **vocabulary/dictionary**

	sentiment	text	tokens_raw	tokens
5464	positive	@VirginAmerica first time flying Virgin, went to #SanFrancisco .Thanks for the smooth ride. Easily my new fav airline!	[@virginamerica, first, time, flying, virgin, ,, went, to, #sanfrancisco, ., thanks, for, the, smooth, ride, ., easily, my, new, fav, airline, !]	[@virginamerica, first, time, fly, virgin, ,, go, to, #sanfrancisco, ., thank, for, the, smooth, ride, ., easily, my, new, fav, airline, !]

Toy Example:

Dataset (or corpus):

Observation 1: "*Time flies like an arrow.*"

Observation 2: "*Fruit flies like a banana.*"

Vocabulary for this dataset:

```
{ time, fruit, flies,                    }
```

Back to our Airline Tweet Data/Model Questions

Q1: Is the data good? Can the tokenization be improved?

Q2: How is our "model"? Can we improve it?

Q3: Is our modeling and model evaluation process okay?
How can it be improved?

ML/Statistical Models: A Typical Example

The following dataset is a list of historical home sale prices and all of the dimensions, features, etc. for each home.

	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	...	GarageArea	OpenPorchSF	MoSold	YrSold	SalePrice
0	60	65.0	8450	7	5	2003	...	548	61	2	2008	208500
1	20	80.0	9600	6	8	1976	...	460	0	5	2007	181500
2	60	68.0	11250	7	5	2001	...	608	42	9	2008	223500
3	70	60.0	9550	7	5	1915	...	642	35	2	2006	140000
4	60	84.0	14260	8	5	2000	...	836	84	12	2008	250000

A company wants to use this data to train a model to be able to predict the sale price of a home well before it is sold (so that the company can purchase it at the optimal price to make a profit after flipping/renting it)

Q: How is this data different from our tweet dataset (after tokenizing)?

Q: What kind of model can be used with this data?

Vectorization: Transforming Tokens to Numbers

Toy Example (pretend that we decided '*a*' and '*an*' are not stop words)

Dataset (or corpus):

Observation 1: "*Time flies like an arrow.*"
Observation 2: "*Fruit flies like a banana.*"

Vocabulary for this dataset:

`{time, fruit, flies, like, a, an, arrow, banana}`

Q: How do we turn this vocabulary into numeric data?

Vectorization: Transforming Tokens to Numbers

Toy Example (pretend that we decided '*a*' and '*an*' are not stop words)

Dataset (or corpus):

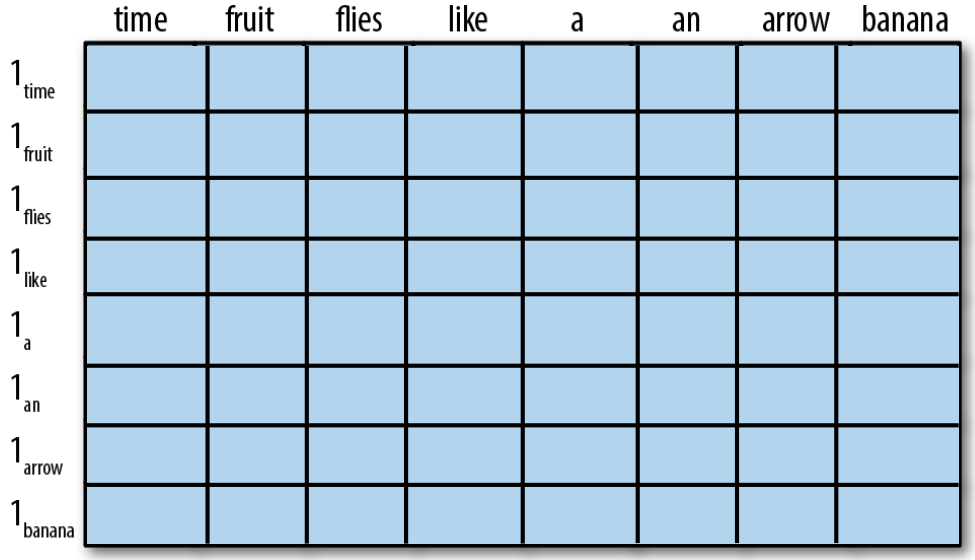
Observation 1: "*Time flies like an arrow.*"
Observation 2: "*Fruit flies like a banana.*"

Vocabulary for this dataset:

`{time, fruit, flies, like, a, an, arrow, banana}`

Q: How can we use this vocabulary to convert observations to numeric data?

A: One-hot Encoding



Vectorization: Transforming Tokens to Numbers

Toy Example (pretend that we decided '*a*' and '*an*' are not stop words)

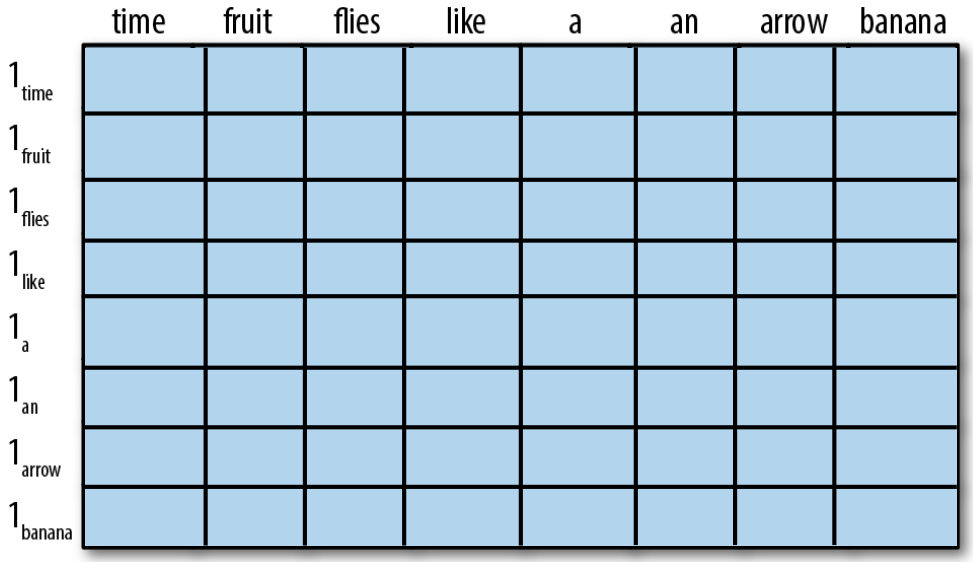
Dataset (or corpus):

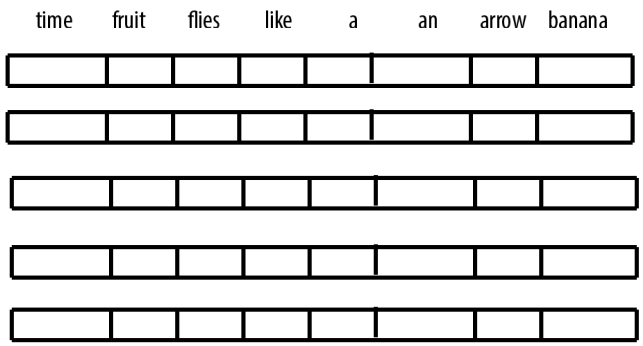
Observation 1: "*Time flies like an arrow.*"
Observation 2: "*Fruit flies like a banana.*"

Vocabulary for this dataset:

`{time, fruit, flies, like, a, an, arrow, banana}`

Q: What are the one-hot encoded vectors for Observation 1?

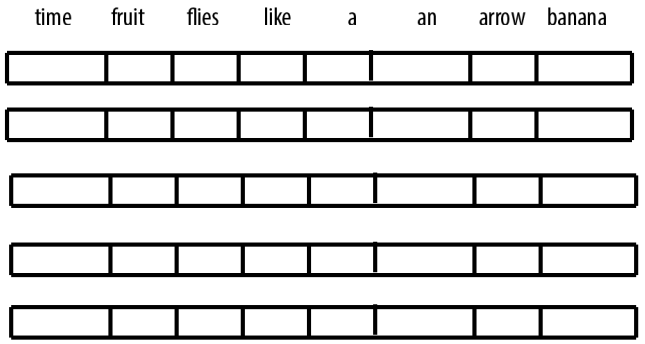




Vectorization: Transforming Tokens to Numbers

Q: How could Observation 1's one-hot vectors be used as input to a model? (e.g. linear regression, random forest, etc.)

Observation 1: "*Time flies like an arrow.*"



Vectorization: Transforming Tokens to Numbers

Dataset (or corpus):

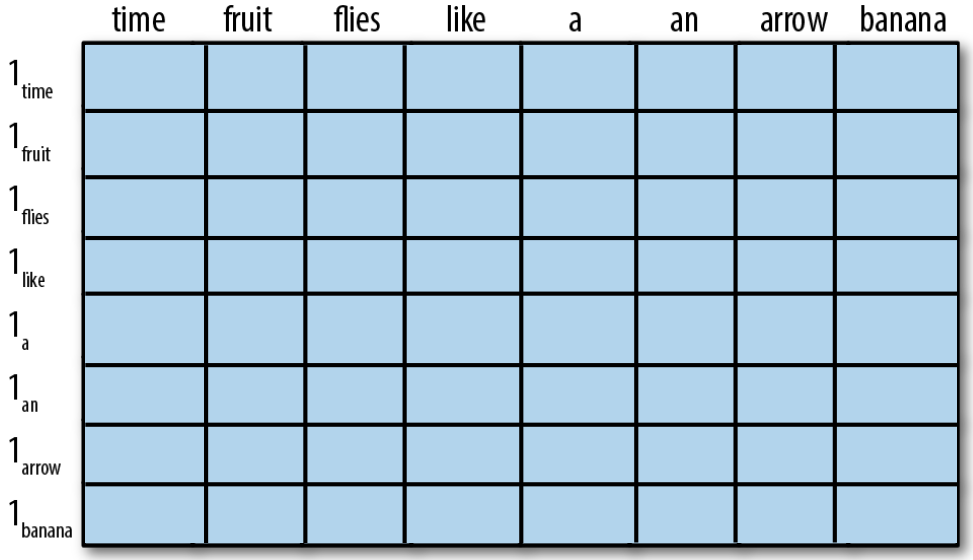
Observation 1: *"Time flies like an arrow."*

Observation 2: *"Fruit flies like a banana."*

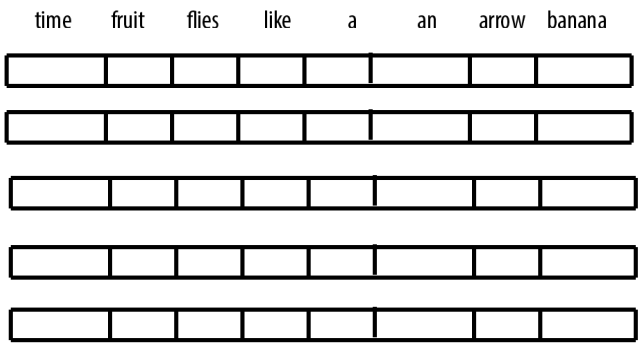
Oftentimes, to reduce input dimensions, each observation is **collapsed**

Collapsed One-hot Encoding: Use logical OR for each one-hot encoded type (or token) in the vocabulary to produce a single vector

Observation 1:



Observation 1 collapsed:



Vectorization: Transforming Tokens to Numbers

Dataset (or corpus):

Observation 1: "*Time flies like an arrow like time.*"

Observation 2: "*Fruit flies like a banana.*"

Vocabulary for this dataset:

`{time, fruit, flies, like, a, an, arrow, banana}`

Q: What information does collapsed one-hot encoding miss out on for the above observations? (aside from the ordering of the words/tokens)

Vectorization: Transforming Tokens to Numbers

Dataset (or corpus):

Observation 1: "*Time flies like an arrow like time.*"

Observation 2: "*Fruit flies like a banana.*"

Vocabulary for this dataset:

`{time, fruit, flies, like, a, an, arrow, banana}`

Term Frequency (TF): Sum one-hot encoded vector for each type (or token) in the vocabulary to produce a single vector

Q: What would vector for Observation 1 look like using term-frequency?

time	fruit	flies	like	a	an	arrow	banana

Vectorization: Transforming Tokens to Numbers

Dataset (or corpus):

Observation 1: "*Time flies like an arrow like time.*"

Observation 2: "*Fruit flies like a banana.*"

Vocabulary for this dataset:

`{time, fruit, flies, like, a, an, arrow, banana}`

Term Frequency-Inverse Document Frequency (TF-IDF): similar to TF representation but also gives more weight to words that are unique to a single document

In addition to TF(w) for each word (and for each document), need to also calculate IDF (w) for each word (across all documents)

TF(w, D) =

IDF(w) =

TFIDF(w, D) = TF(w, D) * IDF(w)

Vectorization: Transforming Tokens to Numbers

Dataset (or corpus):

Observation 1: "*Time flies like an arrow like time.*"

Observation 2: "*Fruit flies like a banana.*"

Vocabulary for this dataset:

`{time, fruit, flies, like, a, an, arrow, banana}`

Term Frequency-Inverse Document Frequency (TF-IDF): similar to TF representation but also gives more weight to words that are unique to a single document

In addition to TF(w) for each word (and for each document), need to also calculate IDF (w) for each word (across all documents)

time fruit flies like a an arrow banana

Vectorization: Transforming Tokens to Numbers

Term Frequency-Inverse Document Frequency (TF-IDF): similar to TF representation but also gives more weight to words that are unique to a single document

In practice, most tools to calculate TF-IDF also use smoothing and normalization

sklearn.feature_extraction.text.TfidfTransformer

class `sklearn.feature_extraction.text.TfidfTransformer`(*, *norm='l2'*, *use_idf=True*, *smooth_idf=True*, *sublinear_tf=False*) [source]

Transform a count matrix to a normalized tf or tf-idf representation.

Tf means term-frequency while tf-idf means term-frequency times inverse document-frequency. This is a common term weighting scheme in information retrieval, that has also found good use in document classification.

The goal of using tf-idf instead of the raw frequencies of occurrence of a token in a given document is to scale down the impact of tokens that occur very frequently in a given corpus and that are hence empirically less informative than features that occur in a small fraction of the training corpus.

The formula that is used to compute the tf-idf for a term t of a document d in a document set is tf-idf(t, d) = tf(t, d) * idf(t), and the idf is computed as idf(t) = log [n / df(t)] + 1 (if `smooth_idf=False`), where n is the total number of documents in the document set and df(t) is the document frequency of t; the document frequency is the number of documents in the document set that contain the term t. The effect of adding "1" to the idf in the equation above is that terms with zero idf, i.e., terms that occur in all documents in a training set, will not be entirely ignored. (Note that the idf formula above differs from the standard textbook notation that defines the idf as idf(t) = log [n / (df(t) + 1)]).

Vectorization: Transforming Tokens to Numbers

Term Frequency-Inverse Document Frequency (TF-IDF): similar to TF representation but also gives more weight to words that are unique to a single document

Ex:
```
corpus = ['Time flies flies like an arrow.',
          'Fruit flies like a banana.']
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
import seaborn as sns

tfidf_vectorizer = TfidfVectorizer()
tfidf = tfidf_vectorizer.fit_transform(corpus).toarray()
sns.heatmap(tfidf, annot=True, cbar=False, xticklabels=vocab,
            yticklabels= ['Sentence 1', 'Sentence 2'])
```

Vectorization: Transforming Tokens to Numbers

Q: What are the various methods of vectorization we could use?

Q: What would be the dimensions of an input for each method?

Q: What are the advantages/disadvantages of each method?

Back to our Airline Tweet Data/Model Questions

Q1: Is the dataset good? Can it be improved?
Can it be converted to a structured and numeric format to be able to be used in other types of models?

Q2: How is our "model"? Can we improve it?
What other types of models could we use?

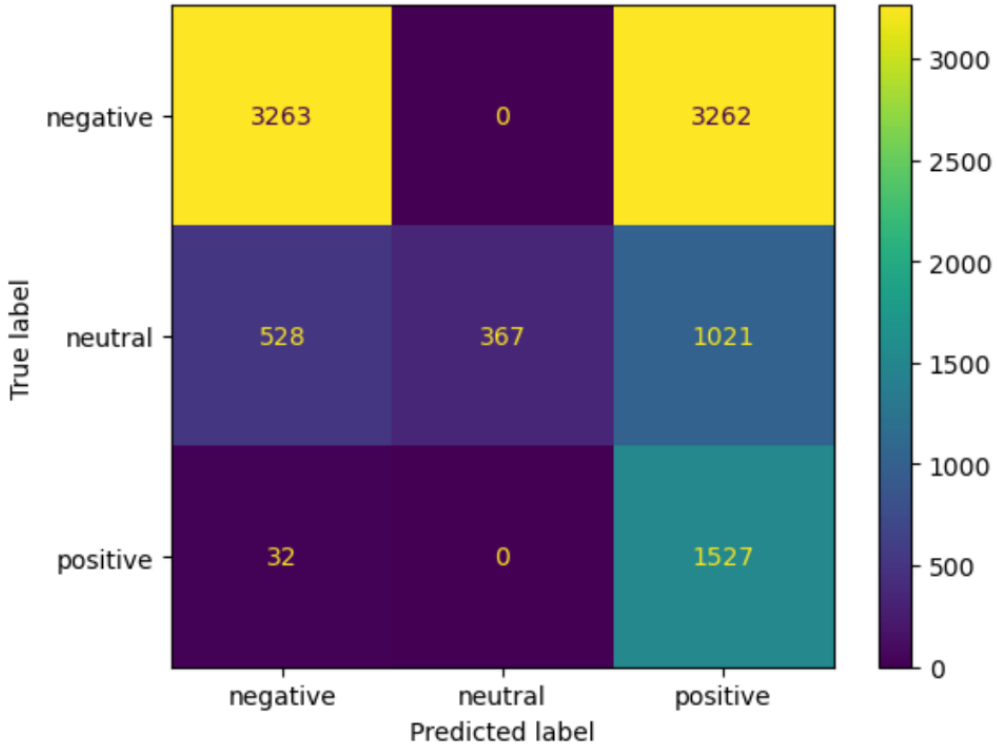
Q3: Is our modeling and model assessment process okay?
How can it be improved?

Model Assessment: Model Performance Metrics

Our Solution: Overall accuracy was ~38% and we looked at accuracy by label with the Confusion Matrix.

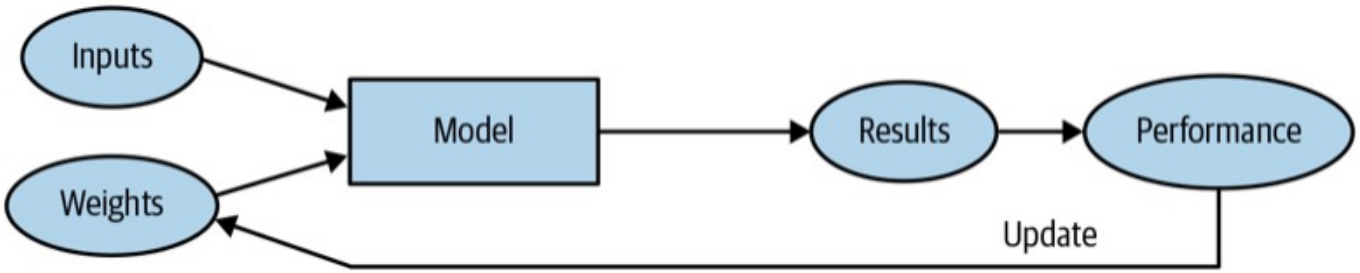
```
from sklearn.metrics import ConfusionMatrixDisplay
disp = ConfusionMatrixDisplay(confusion_matrix(df['sentiment'], df['predicted_sentiment']),
disp.plot()
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7be6cbc5fc10>
```

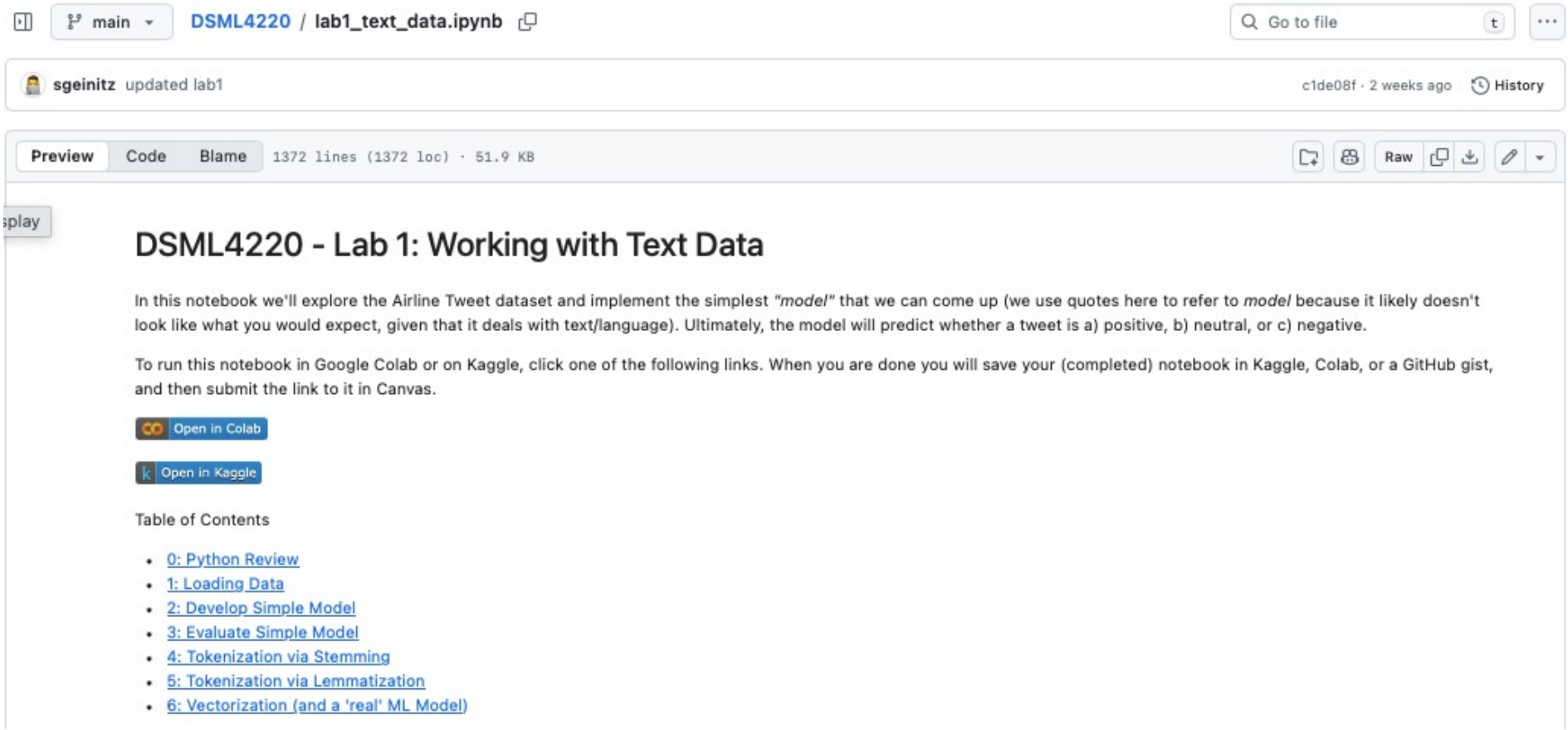


Airline Tweets: Using a 'real' ML Model

Q: What would the inputs look like after using collapsed one-hot encoding? Or, using TF, or TF-IDF?



Airline Tweets: Lab 1



main DSML4220 / lab1_text_data.ipynb

Go to file t

sgeinitz updated lab1 c1de08f · 2 weeks ago History

Preview Code Blame 1372 lines (1372 loc) · 51.9 KB Raw

DSML4220 - Lab 1: Working with Text Data

In this notebook we'll explore the Airline Tweet dataset and implement the simplest *"model"* that we can come up (we use quotes here to refer to *model* because it likely doesn't look like what you would expect, given that it deals with text/language). Ultimately, the model will predict whether a tweet is a) positive, b) neutral, or c) negative.

To run this notebook in Google Colab or on Kaggle, click one of the following links. When you are done you will save your (completed) notebook in Kaggle, Colab, or a GitHub gist, and then submit the link to it in Canvas.

Open in Colab

Open in Kaggle

Table of Contents

- 0: Python Review
- 1: Loading Data
- 2: Develop Simple Model
- 3: Evaluate Simple Model
- 4: Tokenization via Stemming
- 5: Tokenization via Lemmatization
- 6: Vectorization (and a 'real' ML Model)

Intro/Review Machine Learning: Types of Models

ML models fall into various categories based on…

- Task: Classification, Regression, Clustering
- Approach: Supervised, Unsupervised

Models covered in CS 3120

- Linear Models: Linear Regression, Logistic Regression
- Tree-Based Models: Decision Trees, Random Forests
- Distance-Based Models: K-Nearest Neighbors (KNN)
- Probabilistic Models: Naive Bayes
- Clustering Models: K-Means, Hierarchical Clustering
- Neural Networks: Single-layer Perceptron, Multi-layer Perceptron

Intro/Review: Parametric vs Non-parametric Models

Parametric models assume a specific functional form or distribution of the data e.g. linear or quadratic relationships (it summarizes the data through a fixed number of parameters, regardless of the size of the dataset)

- model structure is predefined
- training involves estimating values of a fixed number of parameters
- often requires assumptions about underlying distribution of data

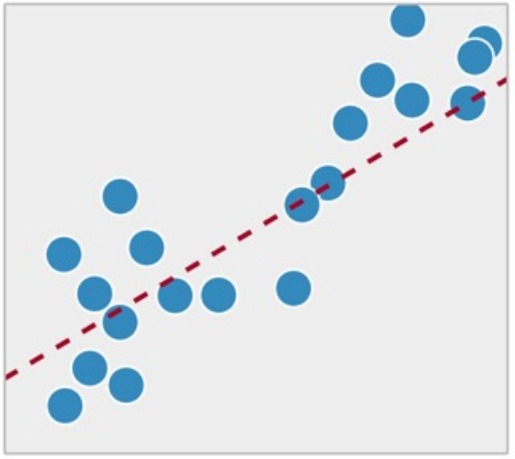
Non-parametric models do not assume any specific functional form nor distribution for the data (instead allows model complexity to grow with the size of the data, meaning number of parameters or structure is not fixed)

- model structure is determined by the data
- model can grow in complexity if more data is available
- no assumption about the distribution of the data, so very flexible

Model	**Param./Non-param.**
Linear Regression	
Logistic Regression	
Decision Trees	
KNN	
Naive Bayes	
Neural Networks	

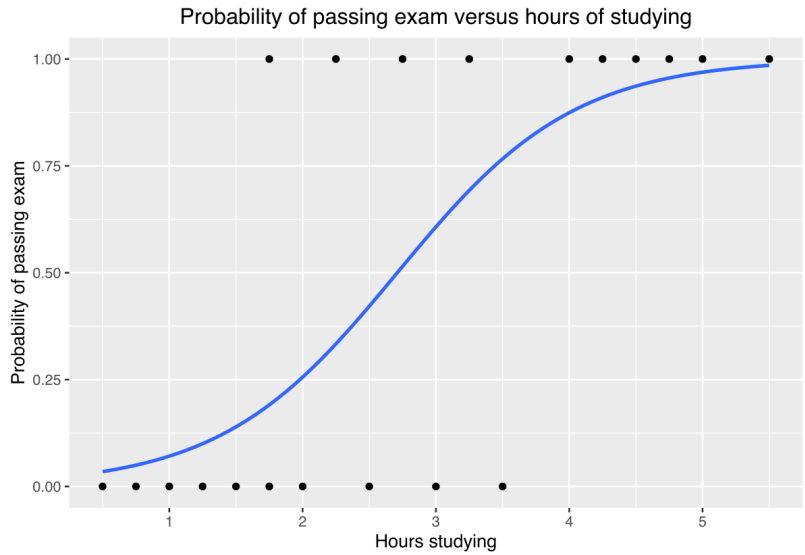
Intro/Review: Machine Learning Models

Q: What are parameters of a least squares regression model?



Intro/Review: Machine Learning Models

Q: What are parameters of a logistic regression (classification) model?



Intro/Review: Machine Learning Models

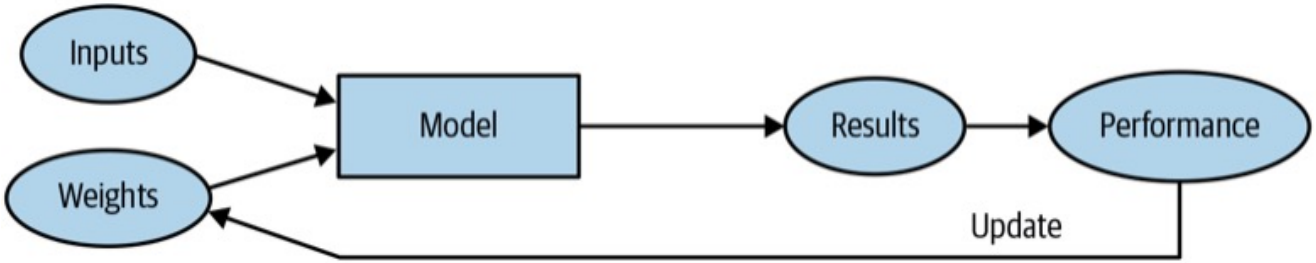
Parametric models assume a specific functional form or distribution of the data ,e.g., linear or quadratic relationships (it summarizes the data through a fixed number of parameters, regardless of the size of the dataset)

- model structure is predefined
- training involves estimating values of a fixed number of parameters
- often requires assumptions about underlying distribution of data

One way to define/describe any parametric model (including neural networks) is as follows.

Given a dataset, D, consisting of n pairs of labels and features, (y_i, x_i), a parametric model f with trainable parameters w can make predictions:

Intro/Review: Machine Learning Overview

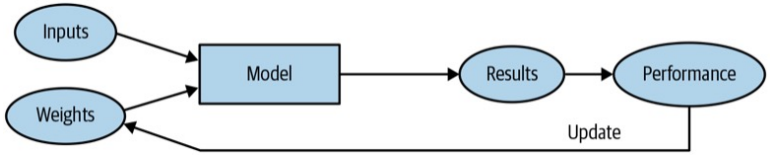


One way to define/describe any parametric model (including neural networks) is as follows.

Given a dataset, D, consisting of n pairs of labels and features, (y_i, x_i), a parametric model f with trainable parameters w can make predictions:

Finding the value of w* is done by minimizing a loss function, L:

ML Intro/Review: Loss Functions



Purpose of loss function is to provide model with feedback based on the current values of the parameter (i.e. weights and biases)

- loss function should be a smooth function that is conducive to numerical (thus, it should vary even if parameter values change only a little bit)

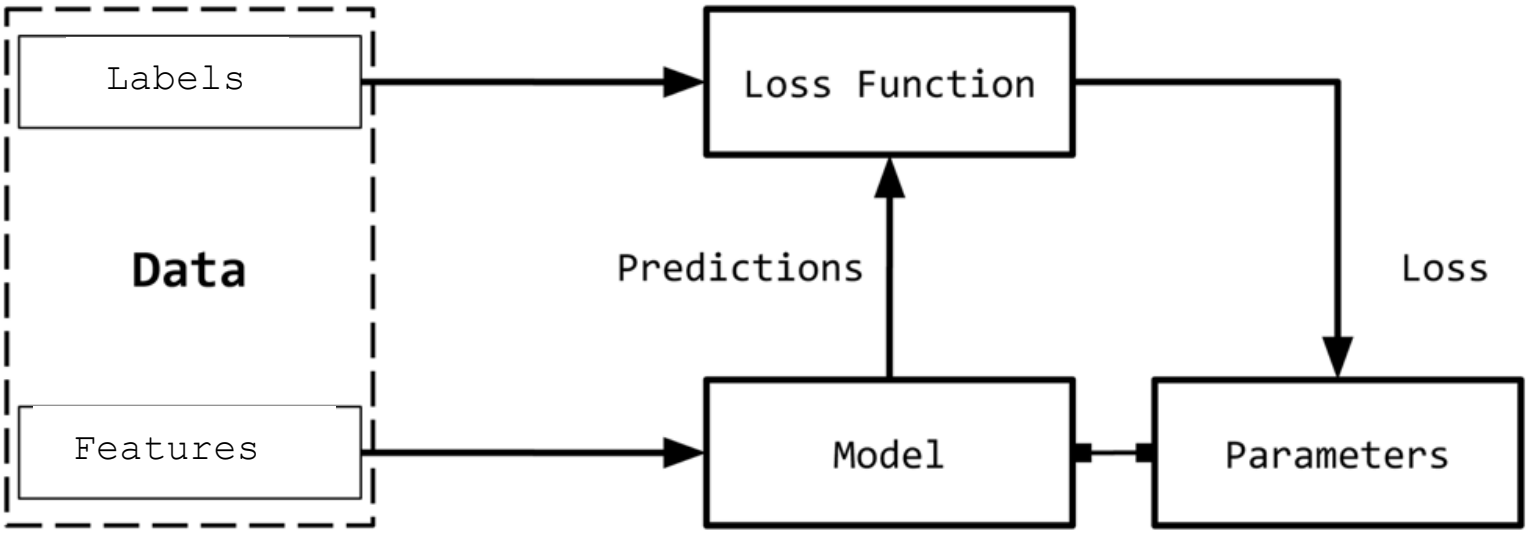
Two general loss functions used, depending on whether it is a regression or classification problem:

Mean-square error: $$L(\vec{w}) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - f\left(\vec{x_i}, \vec{w}\right)\right)^2$$

Cross-entropy: $$L(\vec{w}) = -\sum_{i=1}^{n} y_i \cdot \log\left(f\left(\vec{x_i}, \vec{w}\right)\right)$$

(aka Log-loss)

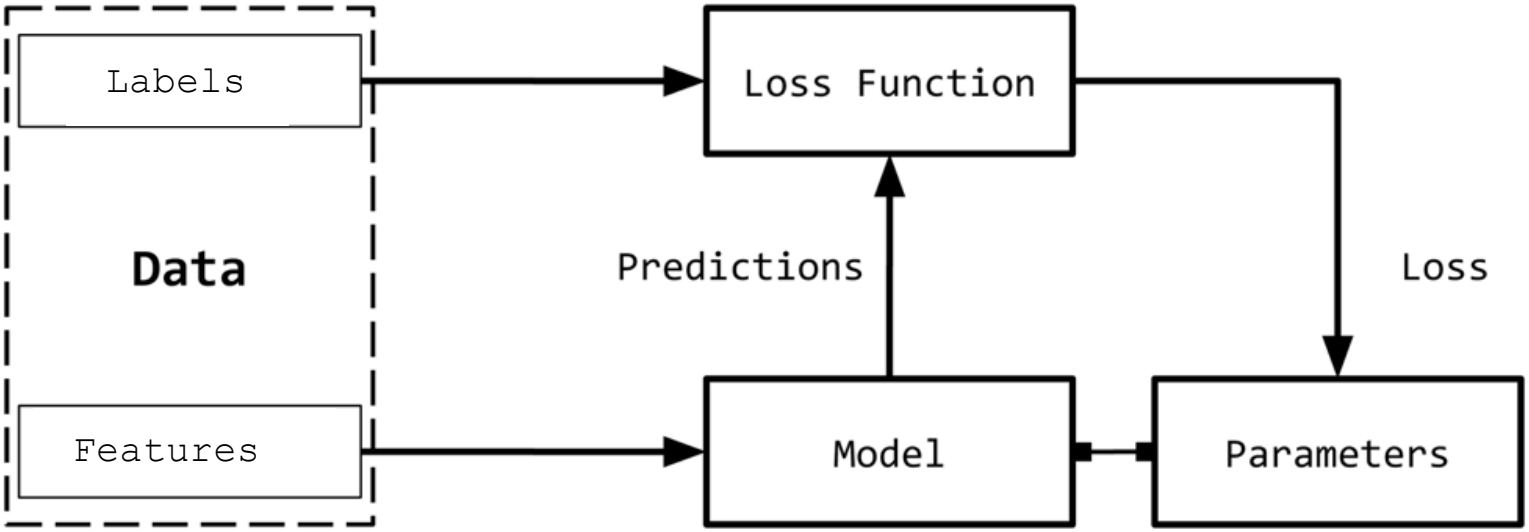
ML Intro/Review: Supervised Learning



Q: What would each component above look like for…

predicting/estimating home price from square footage, # of bedrooms, # of bathrooms, property size, …

ML Intro/Review: Supervised Learning



Q: What would each component above look like for…

predicting survival of a Titanic passenger (yes/no) from, age, cabin number/type, ticket price, …

ML Intro/Review: Performance Metrics

Performance metrics are generally for us (humans) to better understand how "good" a model's predictions are

For regression, the MSE/RMSE is often used (i.e. the loss function itself)

- MSE = $$L(\vec{w}) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - f(\vec{x_i}, \vec{w})\right)^2$$

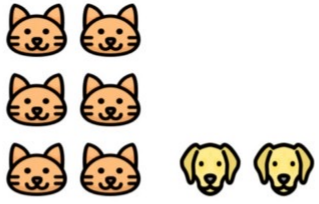
For classification problems the most common metric is:

- Accuracy =

Note: accuracy can be misleading/misused when labels are not balanced

Dataset 1:

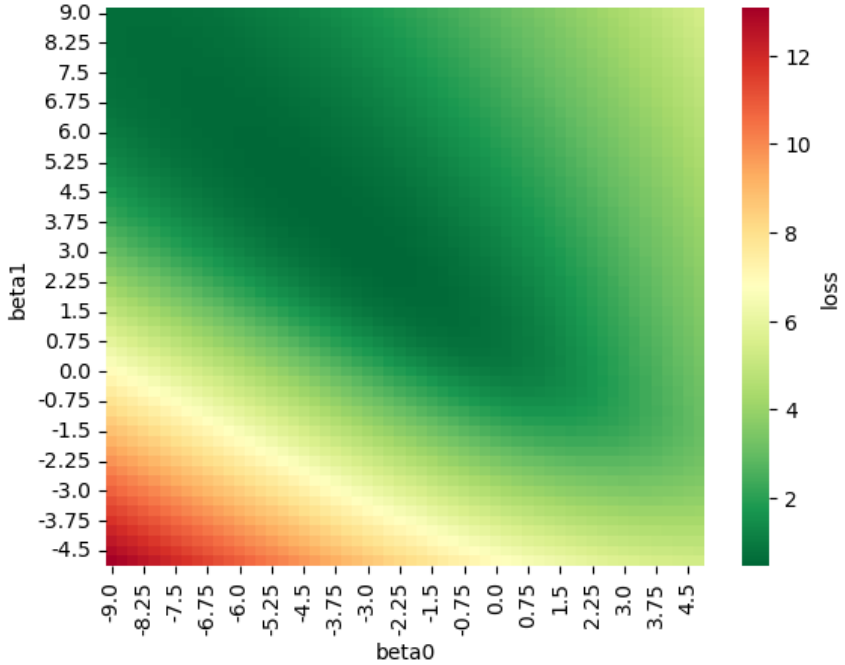
Dataset 2:

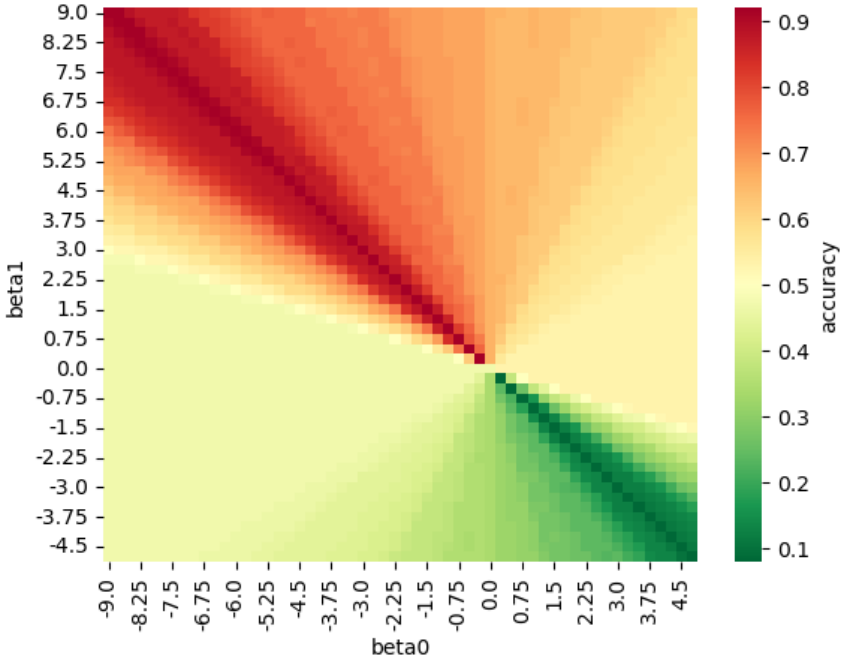


ML Intro/Review: Performance Metrics

Performance metrics are generally for us (humans) to better understand how "good" a model's predictions are

But accuracy and other performance metrics are typically not good as a Loss function for training because often not continuous (i.e. differentiable)



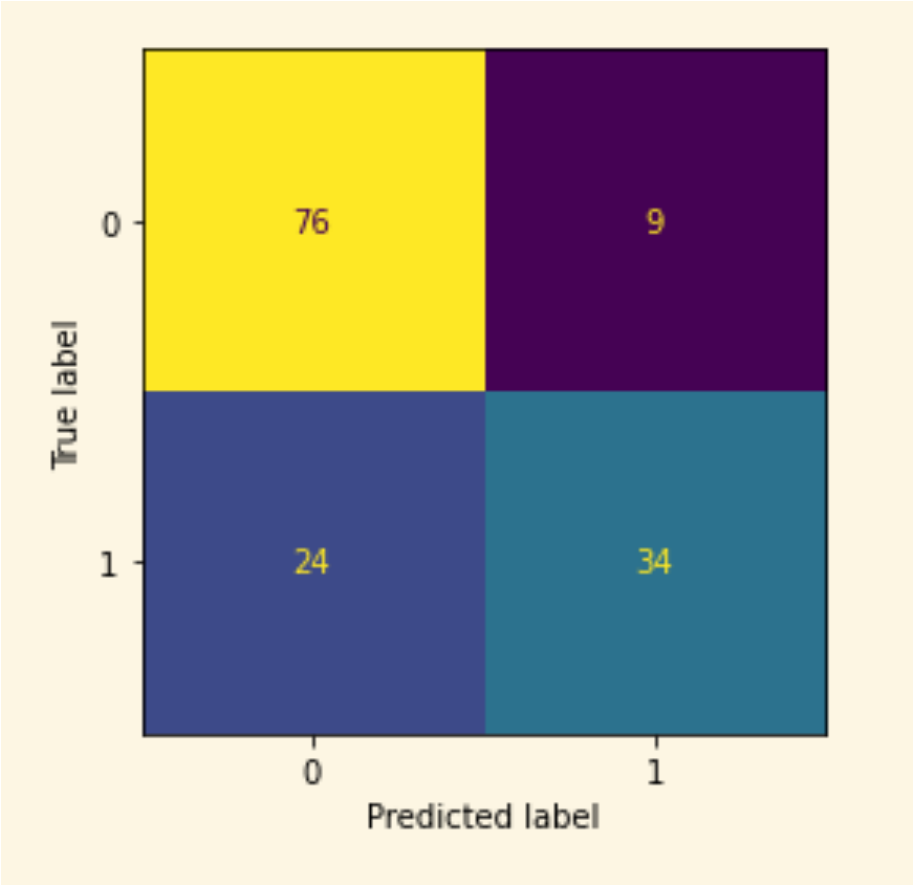


ML Intro/Review: Performance Metrics

Confusion Matrix

- a special type of contingency table used for classification problems
- there is a row and column for each class label with rows denoting actual (or observed) class labels, and columns denoting predicted class labels

Ex:

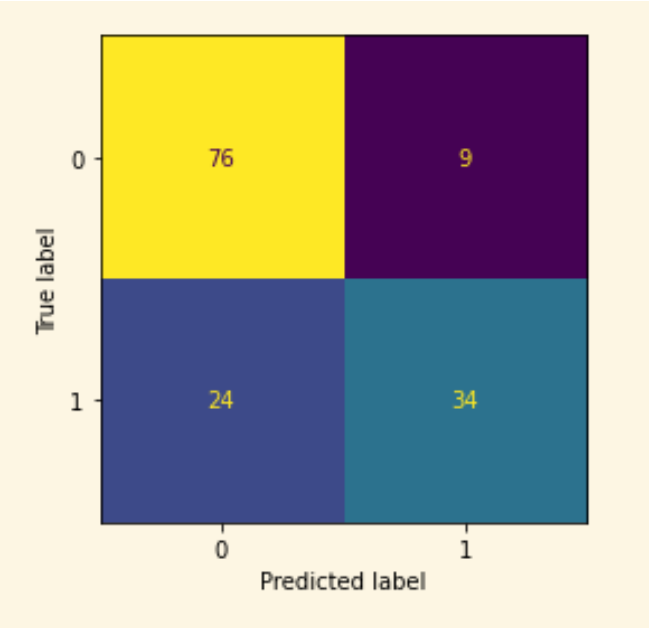


ML Intro/Review: Performance Metrics

Confusion Matrix

- a special type of contingency table used for classification problems
- there is a row and column for each class label with rows denoting actual (or observed) class labels, and columns denoting predicted class labels

A type of confusion matrix is also used in statistical contexts to consider how a statistical test will perform (when carried out many, many times)



Hypothesis testing:

Actual	Decision: H_0 true (Fail to reject)	Decision: H_0 false (Rejecting H_0)
H_0 true	TRUE NEGATIVE Correct decision: **Confidence level** (prob $1-\alpha$)	FALSE POSITIVE Type I Error: **Significance level/Size (α)** (prob α)
H_0 false	FALSE NEGATIVE Type II Error: fail to reject (prob β)	TRUE POSITIVE Correct decision: **Power** (prob $1-\beta$)

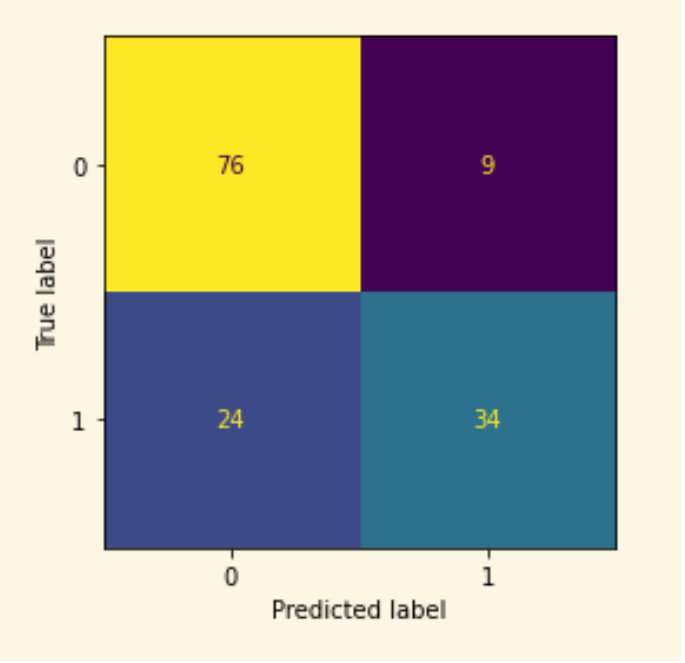
ML Intro/Review: Performance Metrics

Performance metrics are generally for us (humans) to better understand how well a model is able to make predictions

For regression, the MSE/RMSE is often used (i.e. the loss function itself)

For classification problems, common metrics are below (a confusion matrix is helpful to calculate them)

- Accuracy
- Precision
- Recall
- F1 score



ML Intro/Review: Data Types

Primarily three types of data for features

- Numeric (continuous): real-valued data (e.g., age, temperature)
- Ordinal: ordered categories (e.g., ratings: low, medium, high)
- Categorical (nominal): unordered categories (e.g., gender, color)

ML Intro/Review: Data Types

Primarily three types of data for features

- Numeric (continuous): real-valued data (e.g., age, temperature)
- Ordinal: ordered categories (e.g., ratings: low, medium, high)
- Categorical (nominal): unordered categories (e.g., gender, color)

Index	Age	Education	Salary	Department
1	25	High School	45000	Sales
2	32	Bachelor	65000	IT
3	29	Master	70000	HR
4	40	Bachelor	75000	IT
5	22	High School	50000	Marketing

ML Intro/Review: Data Types

Primarily three types of data for features

- Numeric (continuous): real-valued data (e.g., age, temperature)
- Ordinal: ordered categories (e.g., ratings: low, medium, high)
- Categorical (nominal): unordered categories (e.g., gender, color)

ID	Age	Education	Salary	Department	Sales	Marketing	HR	IT
1	25	High School	45000	Sales	1	0	0	0
2	32	Bachelor	65000	IT	0	0	0	1
3	29	Master	70000	HR	0	0	1	0
4	40	Bachelor	75000	IT	0	0	0	1
5	22	High School	50000	Marketing	0	1	0	0

Recap: Tokenization & Vectorization

Tokenization: break down text into smaller units called tokens
(tokens can be words, subwords, characters, or even sentences)

Methods/techniques for tokenization:

-
-

Vectorization: convert tokens into numeric representations that can be processed/understood by an ML model

Methods/techniques for vectorization:

-
-

Recap: Tokenization & Vectorization

Airline Tweets Dataset:

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()
X_train = tfidf_vectorizer.fit_transform(df['textclean']).toarray()
#X_train = tfidf_vectorizer.fit_transform(df['text']).toarray() # original tweet text (without our manual tokenization)

print(f"X_train.shape = {X_train.shape}")
type(X_train)
```

✓ 0.1s

```
X_train.shape = (8500, 7480)

numpy.ndarray
```

Recap: Tokenization & Vectorization

Airline Tweets Dataset:

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()
X_train = tfidf_vectorizer.fit_transform(df['textclean']).toarray()
#X_train = tfidf_vectorizer.fit_transform(df['text']).toarray() # original tweet text (without our manual tokenization)

print(f"X_train.shape = {X_train.shape}")
type(X_train)
```

✓ 0.1s

```
X_train.shape = (8500, 7480)

numpy.ndarray
```

One observation:

```
obs1 = list(X_train[0,:])
for i, tfidf_val in enumerate(obs1):
    if tfidf_val > 0:
        print(f"obs1 has a non-zero TF-IDF value: {tfidf_val} at col i={i} (associated with token: {tfidf_vectorizer.get_fea
```

✓ 0.0s

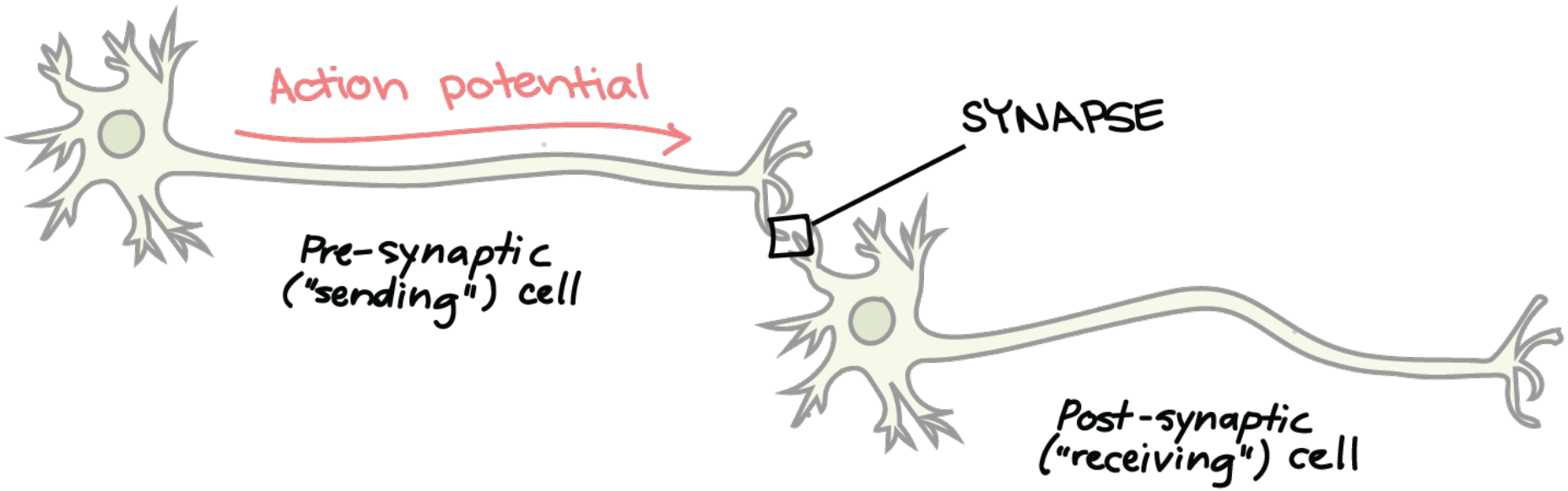
```
obs1 has a non-zero TF-IDF value: 0.234628705023222 at col i=1170 (associated with token: airport)
obs1 has a non-zero TF-IDF value: 0.404389847999136 at col i=1561 (associated with token: beach)
obs1 has a non-zero TF-IDF value: 0.2578032056127592 at col i=2120 (associated with token: come)
obs1 has a non-zero TF-IDF value: 0.4669179666423831 at col i=2560 (associated with token: destin)
obs1 has a non-zero TF-IDF value: 0.39184619945308913 at col i=3268 (associated with token: fort)
obs1 has a non-zero TF-IDF value: 0.30429929734595496 at col i=5441 (associated with token: provide)
obs1 has a non-zero TF-IDF value: 0.18107856651785056 at col i=6016 (associated with token: service)
obs1 has a non-zero TF-IDF value: 0.4669179666423831 at col i=7213 (associated with token: walton)
```

Neural Networks

Early Machine Learning: Biological Inspiration

"When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased."

– Donald Hebb
(psychologist/neuropsychologist)

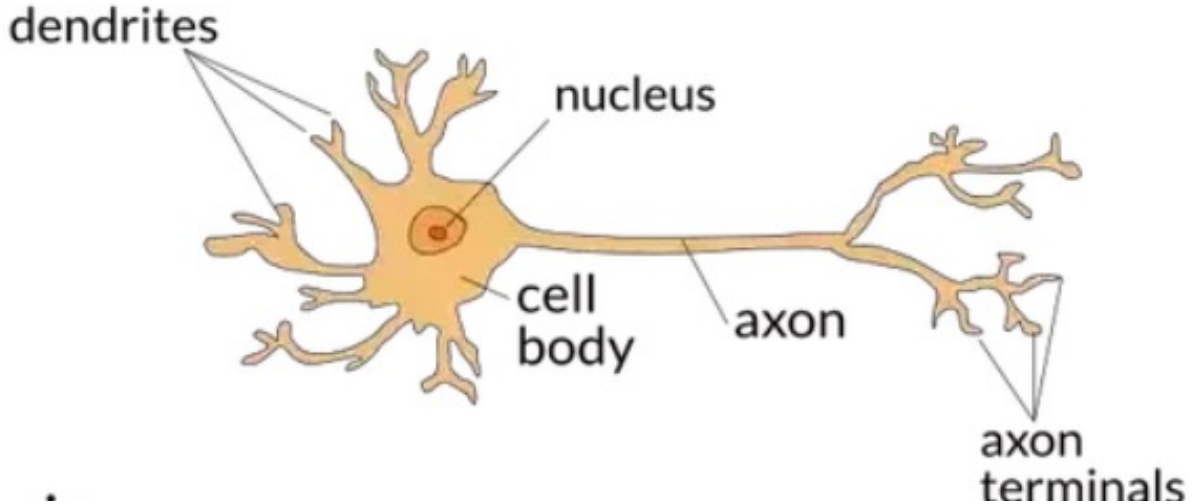


Early Machine Learning: Perceptron Model

Artificial Neural Network (ANN) – based on a biological neuron

- Proposed in 1943 by McCulloch and Pitts, "*A Logical Calculus of the Ideas Immanent in Nervous Activity*":

  "Because of the 'all-or-none' character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms."

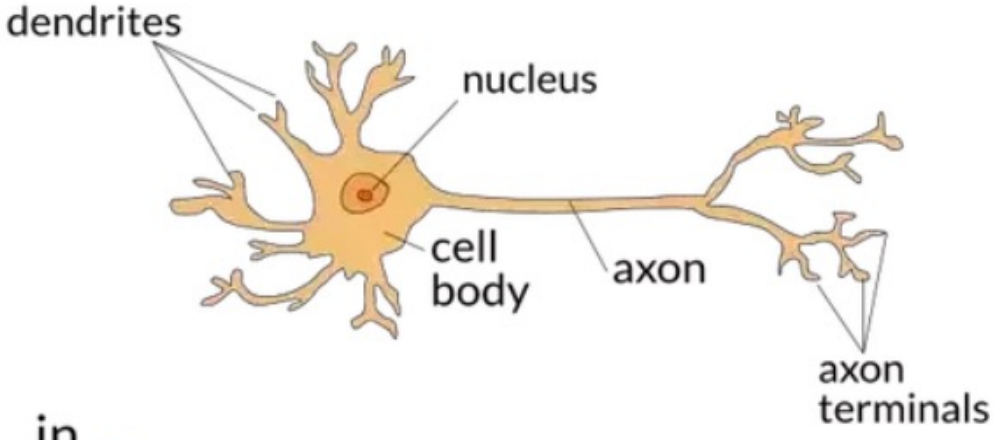


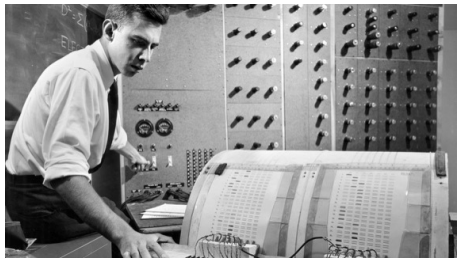
Early Machine Learning: Perceptron Model

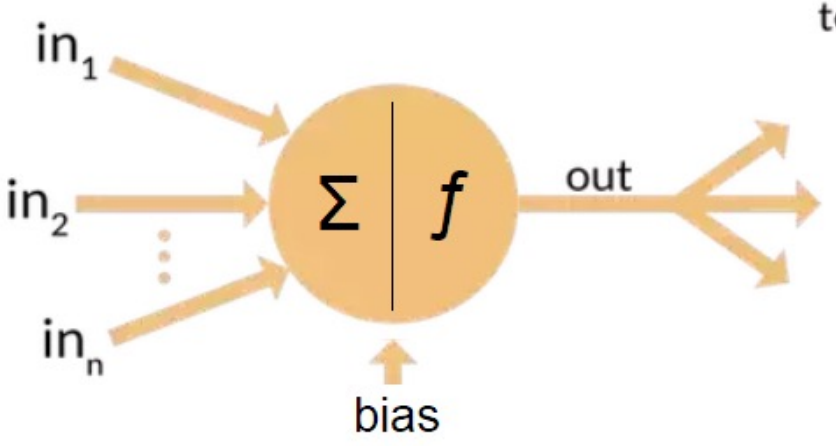
Work later picked up by Frank Rosenblatt

- Mark I Perceptron (a working computer/device based on an ANN)

> *"We are about to witness the birth of such a machine – a machine capable of perceiving, recognizing and identifying its surroundings without any human training or control."*
>
> - Rosenblatt (1958)







Early Machine Learning: Coining the Term

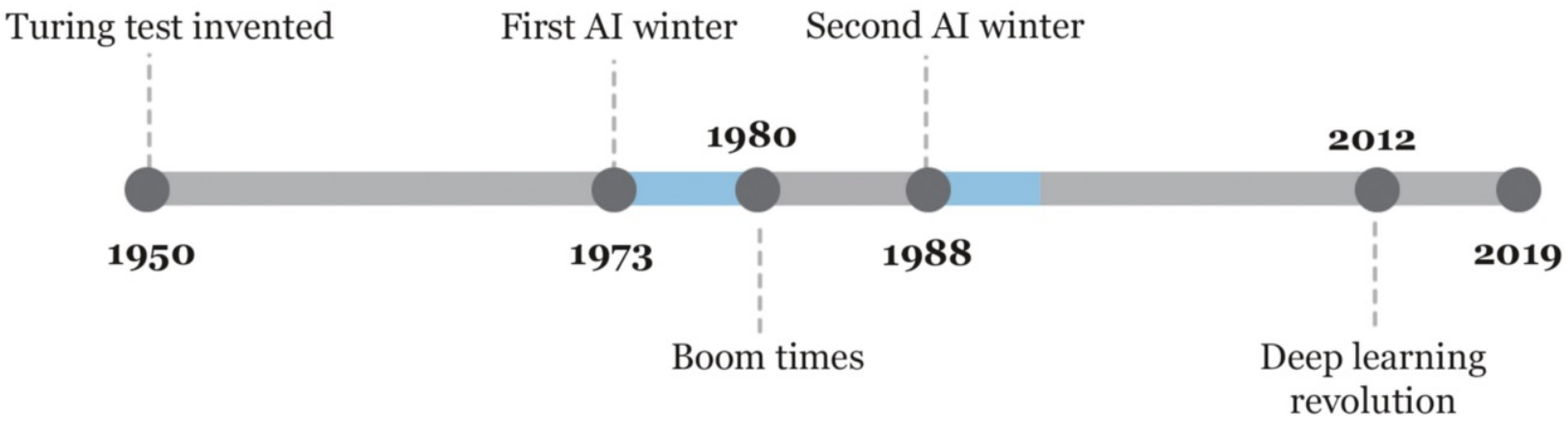
Although used earlier, Arthur Samuel, popularized the term, “Machine Learning”, and developed a self-learning checkers-playing program (was not based on a neural network though!)

Arthur Lee Samuel *(1959)*

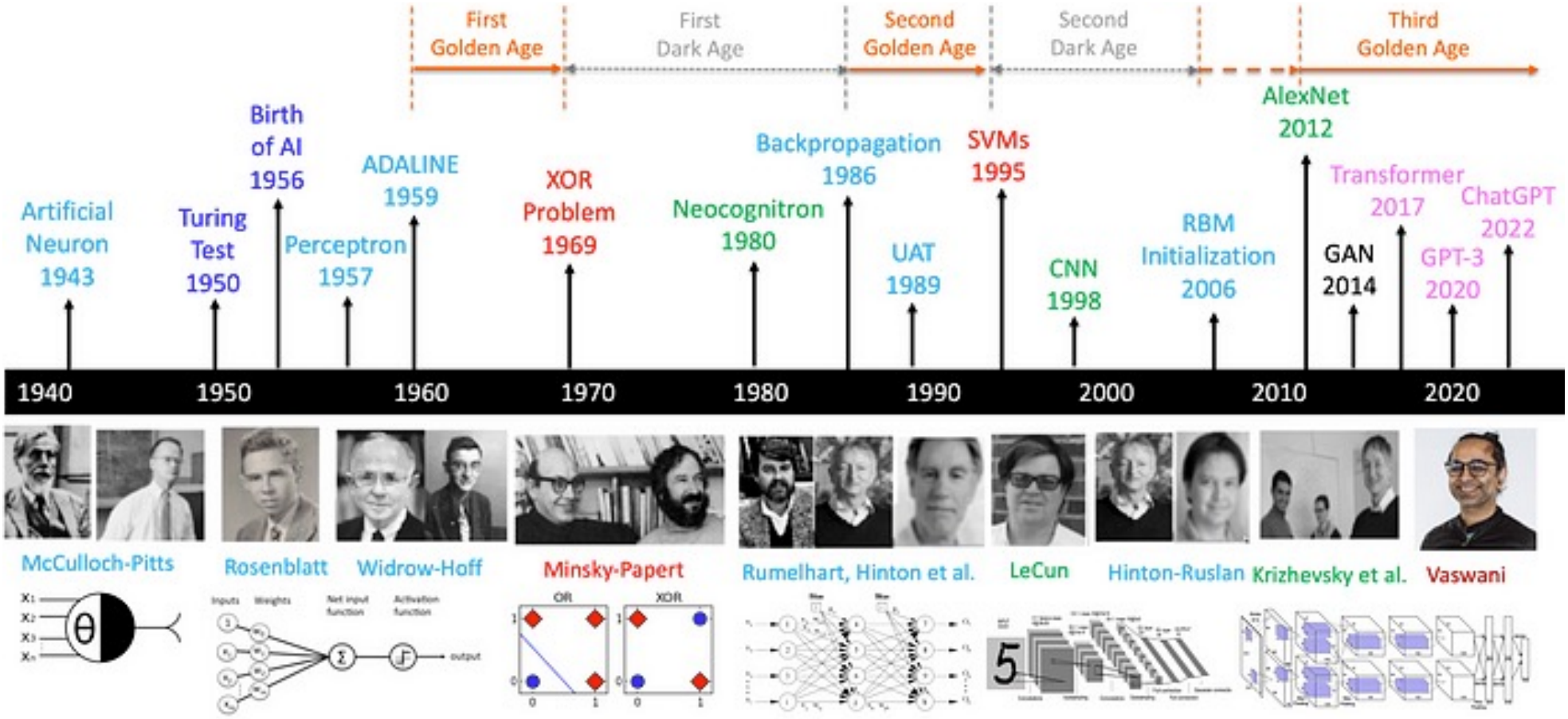
Machine Learning *the "field of study that gives computers the ability to learn without being explicitly programmed".*

History of Artificial Intelligence: A Timeline

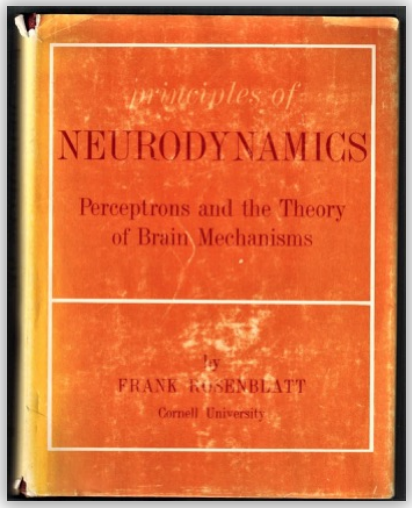
Def: AI Winter – a stagnant, quiet period for research and development of AI due to lack of funding from governments and businesses, and customer interest



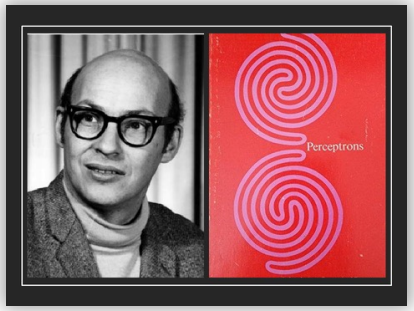
History of Artificial Intelligence: A (better) Timeline

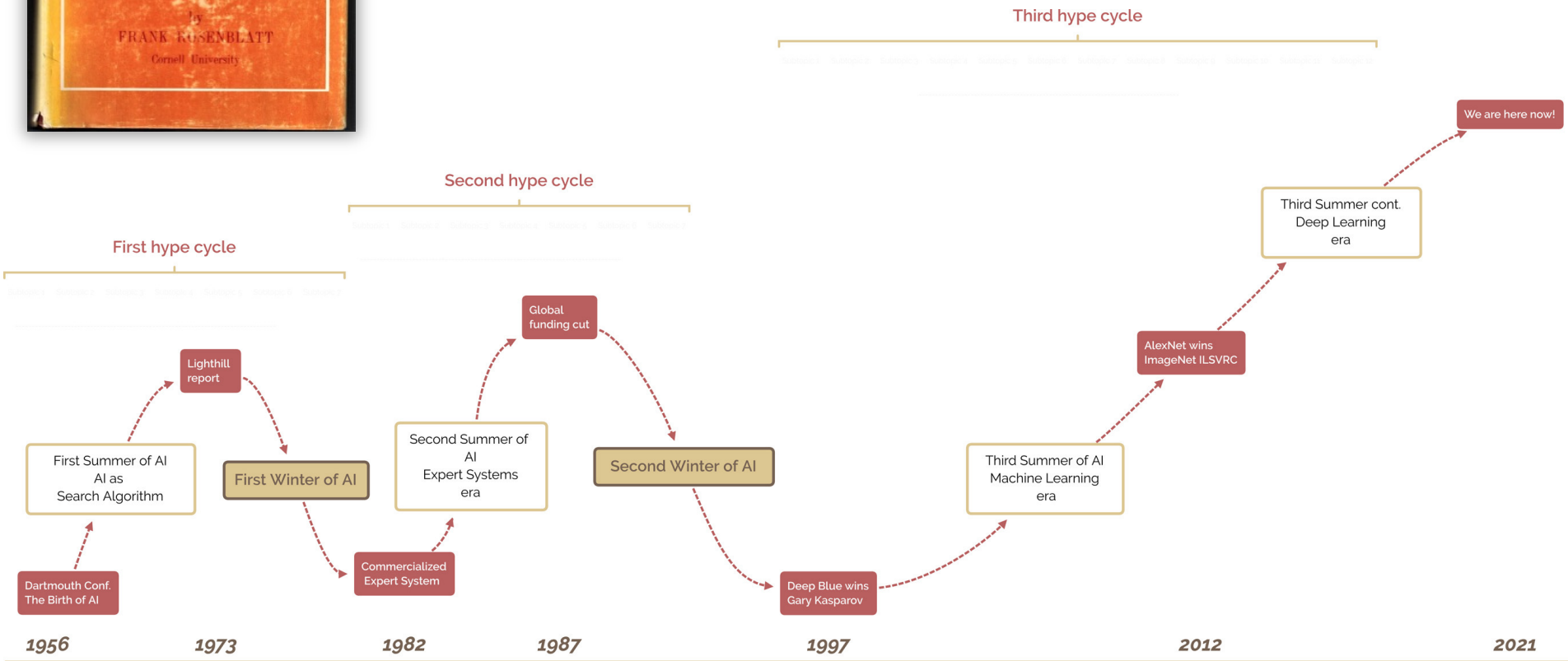


History of Artificial Intelligence: A(nother) Timeline



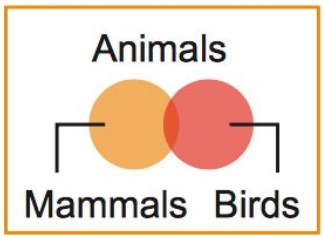
vs.





History of Artificial Intelligence: Five Tribes

Symbolists



Use symbols, rules, and logic to represent knowledge and draw logical inference

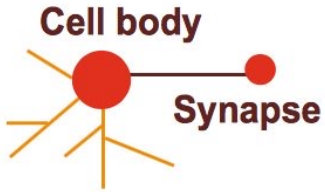
Favored algorithm
Rules and decision trees

Bayesians

Assess the likelihood of occurrence for probabilistic inference

Favored algorithm
Naive Bayes or Markov

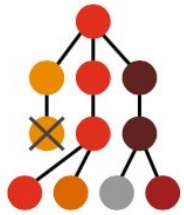
Connectionists



Recognize and generalize patterns dynamically with matrices of probabilistic, weighted neurons

Favored algorithm
Neural networks

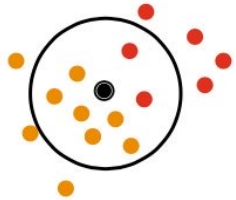
Evolutionaries



Generate variations and then assess the fitness of each for a given purpose

Favored algorithm
Genetic programs

Analogizers



Optimize a function in light of constraints ("going as high as you can while staying on the road")

Favored algorithm
Support vectors

Neural Networks: Structure of an Bio-NN

It's not just one, but many neurons that must be combined/processed

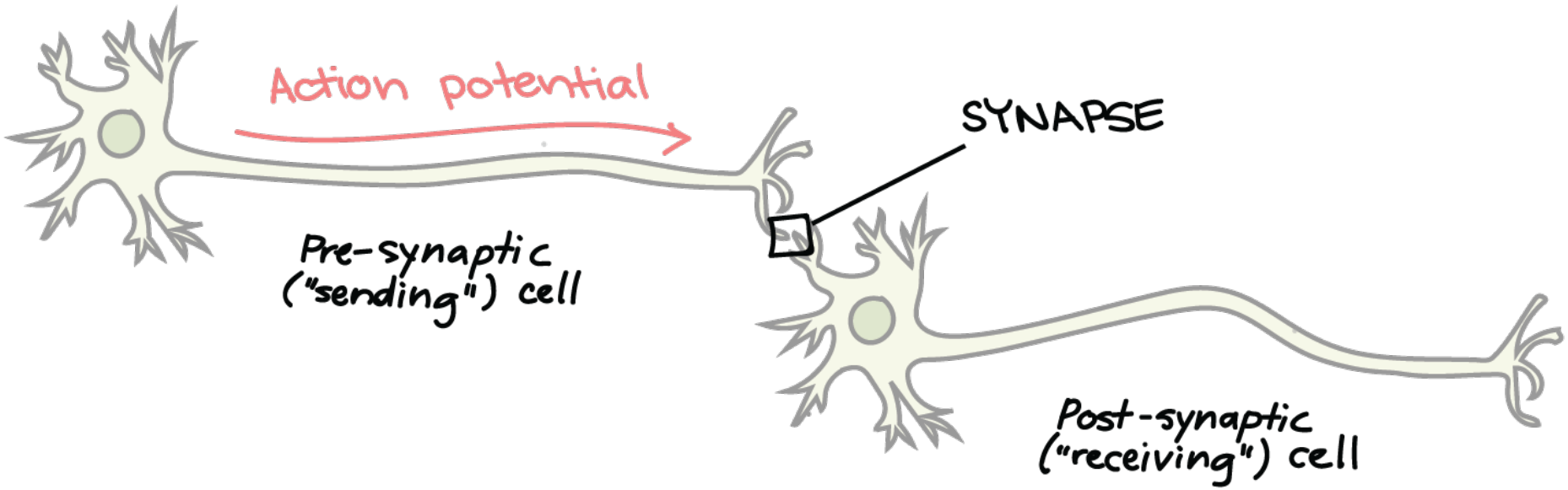
(combining the neuron's **memory** with the **stimulation** sent to it)

to then determine if the **system** should:

- activate the neuron in the next layer

  OR

- trigger a final response, or not



Neural Networks: Structure of an Bio-NN

It's not just one, but many neurons that must be combined/processed

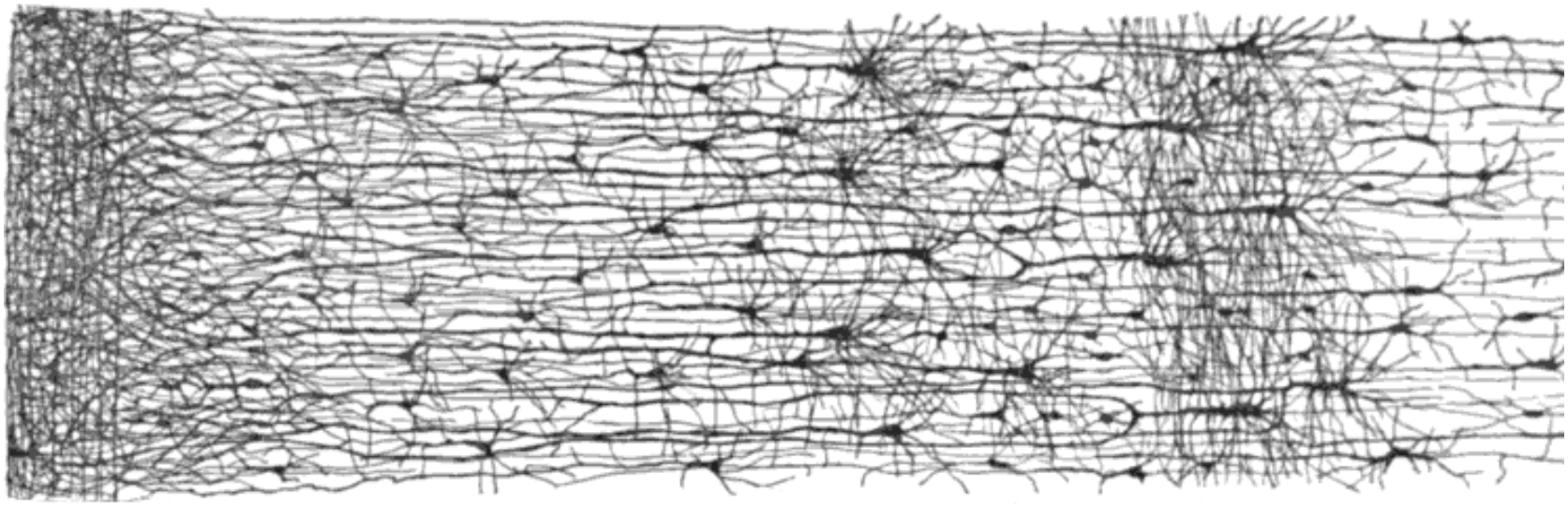
(combining the neuron's **memory** with the **stimulation** sent to it)

to then determine if the **system** should:

- activate the neuron in the next layer

  OR

- trigger a final response, or not



Neural Networks: Structure of an ANN

It's not just one, but many neurons that must be combined/processed

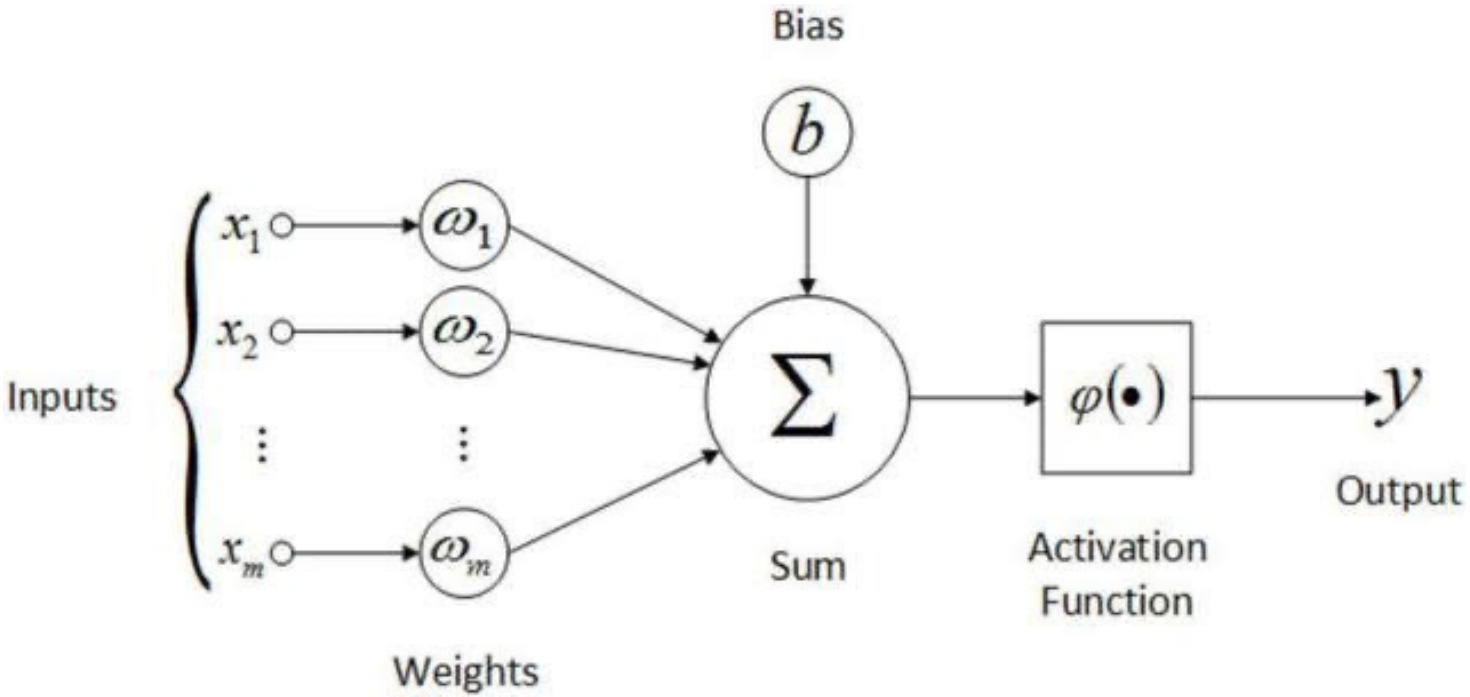
(combining the neuron's **weight parameter** with the **x input** sent to it)

to then determine if the **model** should:

- activate the neuron in the next layer **(in a multi-layer neural network)**

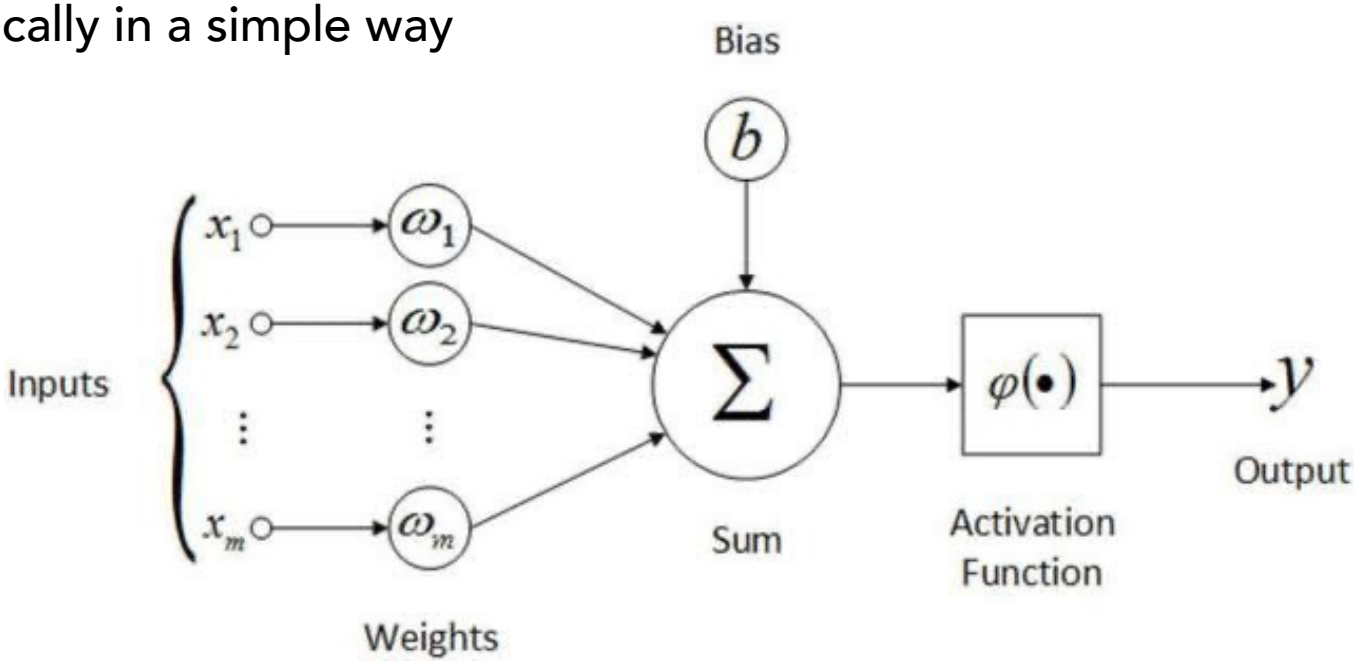
  OR

- **predict a positive label, or negative label (in the final layer)**



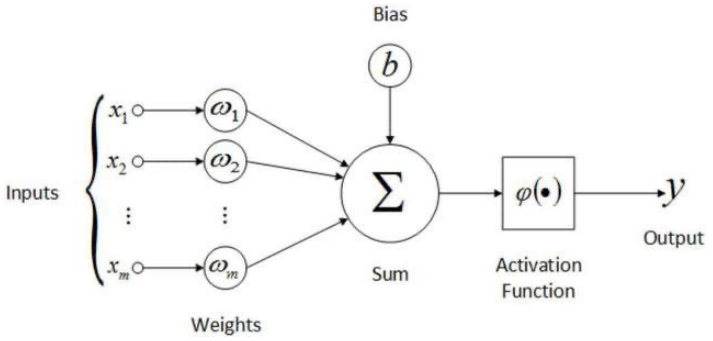
Neural Networks: Structure of an ANN

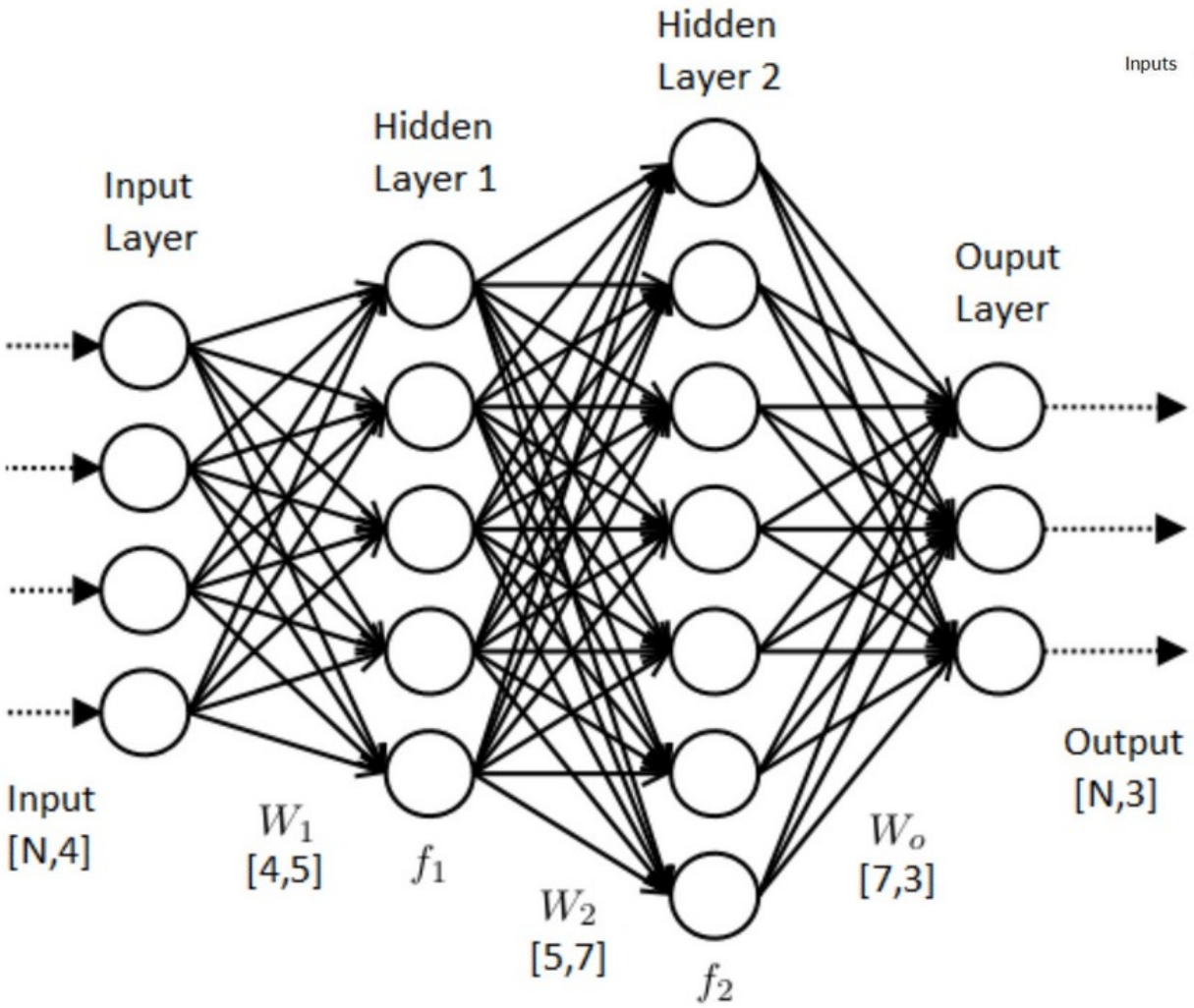
Despite the complex appearance and structure, it's possible to express mathematically in a simple way



Neural Networks: Structure of an ANN

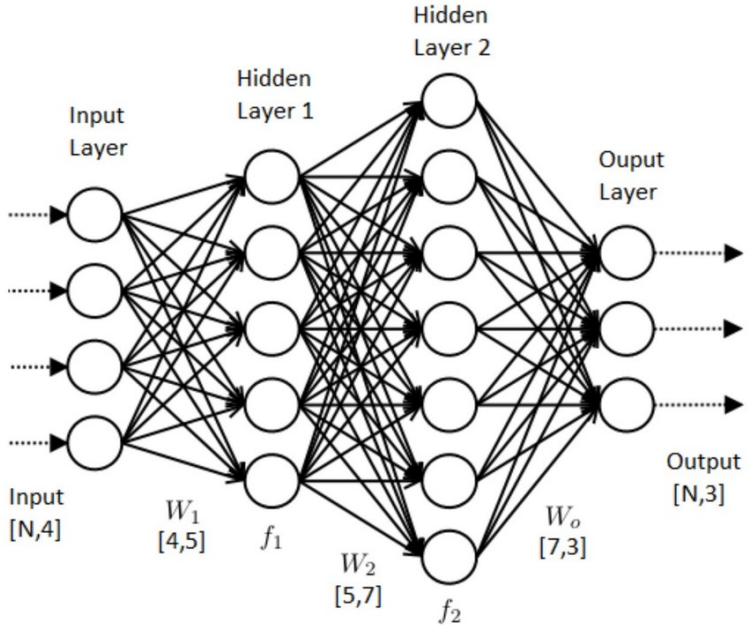
Multi-layer networks behave the same, just with this same model repeated many, many times





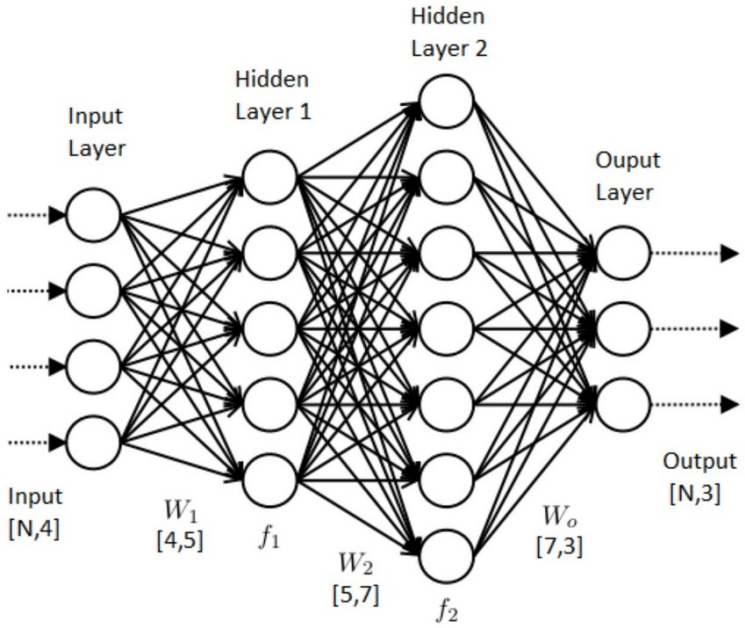
Neural Networks: Structure of a ANN

Q: How many parameters are in this neural network model?



Neural Networks: Activation Functions

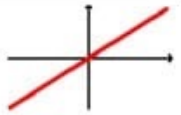
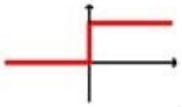
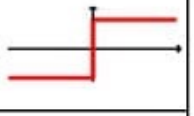
Q: Why do we need activation functions?
(i.e. what happens if we don't use them between layers?)



Neural Networks: Activation Functions

Activation functions are important because they allow for non-linearities, which are able to learn complex patterns/relationships.

(without them a neural network could be stated as a simple linear model)

Activation Function	Equation	Example	1D Graph
Linear	$\phi(z) = z$	Adaline, linear regression	
Unit Step (Heaviside Function)	$\phi(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Sign (signum)	$\phi(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Piece-wise Linear	$\phi(z) = \begin{cases} 0 & z \leq -\frac{1}{2} \\ z + \frac{1}{2} & -\frac{1}{2} \leq z \leq \frac{1}{2} \\ 1 & z \geq \frac{1}{2} \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multilayer NN	
Hyperbolic Tangent (tanh)	$\phi(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multilayer NN, RNNs	
ReLU	$\phi(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases}$	Multilayer NN, CNNs	

Neural Networks: A Simple Perceptron Model

Suppose we want to build a small, simple, neural network with one perceptron.

Q: How do we learn whether to 'activate' the neuron (based on some input, x)?

Neural Networks: A Simple Perceptron Model

Suppose we want to build a small, simple, neural network with one perceptron.
Q: How do we learn whether to 'activate' the neuron (based on some input, x)?

Specifically:

Q: What does the neural network look like?

Q: How do we train/fit this model? What loss function to use? etc.

Neural Networks: Structure of a ANN

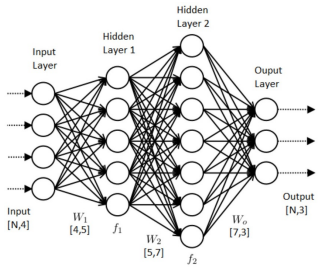
Q: How many parameters would be in the neural network for the Airline Tweet dataset?

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()
X_train = tfidf_vectorizer.fit_transform(df['textclean']).toarray()
#X_train = tfidf_vectorizer.fit_transform(df['text']).toarray() # original tweet text (without our manual tokenization)

print(f"X_train.shape = {X_train.shape}")
type(X_train)
```

✓ 0.1s

```
X_train.shape = ( 8500 , 7480 )
```



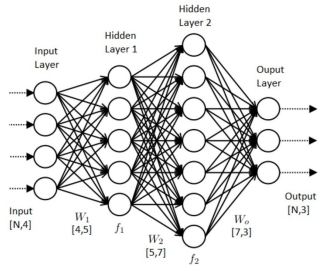
Neural Networks: Structure of a ANN

Q: How many parameters would be in the neural network for the Airline Tweet dataset if there were 2x as many tweets and from many different regions (with different dialects/languages)?

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()
X_train = tfidf_vectorizer.fit_transform(df['textclean']).toarray()
#X_train = tfidf_vectorizer.fit_transform(df['text']).toarray() # original tweet text (without our manual tokenization)

print(f"X_train.shape = {X_train.shape}")
type(X_train)
```

✓ 0.1s



X_train.shape = (,)

Today

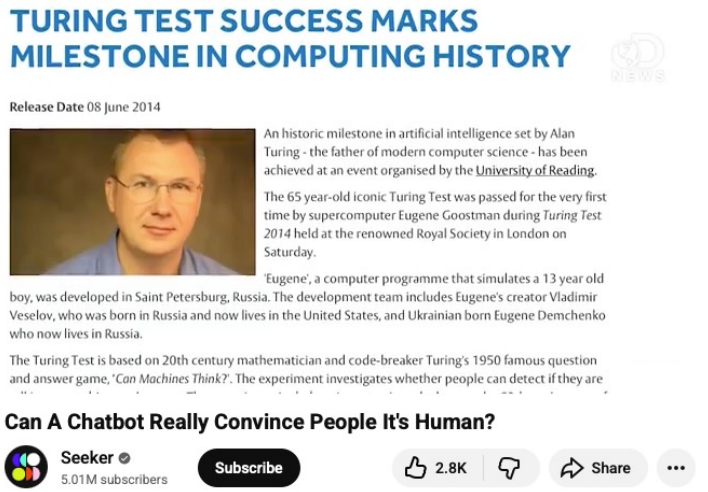
- Finish w/ Airline Tweets, and Tokenization & Vectorization
- ML Review – Models, Metrics, Neural Networks

To Do

- Discussion 1 (due today)
- Lab 1 (Sat, Jan 31st)
- Kaggle 1 (Mon, Feb 2nd)
- Discussion 2 (Sat, Feb 7th)

Next Time

- Bias-Variance Tradeoff
- Parameters vs Hyperparameters
- Numpy and PyTorch (Broadcasting and Optimization)
- NN to Predict Surname Nationality



https://youtu.be/njmAUhUwKys

https://youtu.be/zhxNI7V2IxM

'ELIZA,' the world's 1st chatbot, was just resurrected from 60-year-old computer code

News By Kristina Killgrove published January 17, 2025

Researchers discovered long-lost computer code and used it to resurrect the early chatbot ELIZA.

https://www.livescience.com/technology/eliza-the-worlds-1st-chatbot-was-just-resurrected-from-60-year-old-computer-code